
ORD-Bench: An Adversarially Constructed Benchmark for Enterprise Resource Selection

Dominik Neumaier^{*1}

¹University of Applied Sciences Karlsruhe & University of Strasbourg

Correspondence: nedo1011@h-ka.de

Abstract

LLM-based agents invoking resources in enterprise landscapes must distinguish the right resource from many similar alternatives, a disambiguation problem bounded by how resources are described. Typed resource description standards such as Open Resource Discovery (ORD) promise richer, semantically enhanced and machine-readable descriptions compared to the free text descriptions agents often rely on today, enabling agents to leverage typed fields for disambiguation and reliable orchestration. Yet whether this structure and its semantic enrichment can actually reduce ambiguity in a dense resource landscape has not been measured, as no benchmark provides structured enterprise resources with controlled ambiguity and matched enriched and non-enriched descriptions. This paper introduces **ORD-Bench**, a benchmark of 273 enterprise resources across ten systems, built for difficulty rather than scale. Its contributions are: (1) an enterprise resource landscape represented natively in ORD; (2) a field-based ambiguity metric combined with an adversarial construction loop that generates controlled ambiguity rather than relying on catalogue size; (3) 30 process models with derived skill artefacts producing semantic enrichment while preserving the resource set; and (4) 350 test cases as a reusable substrate. Furthermore, a pairwise ambiguity analysis finds that semantic enrichment lowers structural ambiguity by up to 28.3% on the most confusable pairs, yet *raises* embedding-based similarity by 27.7%, indicating that ambiguity is a property of the retrieval representation rather than the resource. All artefacts are released openly.

1. Introduction

LLM-based agents are increasingly being adopted to automate enterprise processes by autonomously reasoning about tasks, selecting appropriate resources, and invoking capabilities across heterogeneous system landscapes [1, 2]. As enterprise resource landscapes grow, reliably identifying the appropriate resource becomes a disambiguation problem: resources across different systems, vendors, and business domains may expose similar terminology, capabilities, or contextual information, requiring agents to distinguish suitable resources from similar alternatives [3]. Selecting an appropriate resource therefore requires more than matching technical interfaces or function names; it requires understanding the business intent of a task together with the context in which a resource is applicable. Consequently, the structure and semantic richness of resource descriptions become critical factors for resource discovery and selection.

Recent developments in resource description standards provide a foundation for representing enterprise resources through structured, machine-readable metadata. Unlike traditional interface descriptions, these approaches represent resources through typed structures that capture both technical properties and semantic information relevant for discovery. Open Resource Discovery (ORD) [4] introduces such a description model for heterogeneous enterprise resources, while Google’s Agent Resource Discovery (ARD) [5] addresses the discovery and cataloguing of agentic resources across organizational boundaries. However, adoption of these standards is still emerging, and publicly available typed resource collections remain scarce.

Consequently, the influence of structured descriptions and

additional semantic information on resource distinguishability cannot be systematically evaluated. Doing so requires enterprise resources in a structured format, a controlled generation process that systematically places ambiguous alternatives next to each target, and the ability to compare identical resources under different description conditions.

Existing benchmarks address important aspects of agentic resource selection, including scalability and general-purpose API coverage [6–9], but none satisfies these requirements together. Following the Design Science Research paradigm of Österle et al. [10], this work addresses that gap; its central research question is: *How can a benchmark for enterprise resources be constructed that makes the influence of structured and semantically enriched descriptions on resource distinguishability measurable?*

ORD-Bench addresses this gap. It contributes: (1) a typed enterprise resource landscape of 273 resources across ten systems, represented natively in ORD; (2) a field-based ambiguity metric combined with an adversarial construction loop that generates controlled ambiguity rather than relying purely on benchmark size; (3) 30 enterprise process models with derived skill artefacts that provide additional semantic resource context while preserving the underlying resource set, enabling controlled comparison of different resource description conditions; and (4) 350 test cases across these conditions as a reusable substrate for future LLM-based resource retrieval and selection studies.¹

This work focuses on benchmark construction and validation rather than evaluating individual resource retrieval

¹<https://github.tools.sap/I750252/MasterThesis>

strategies. As an initial validation, it demonstrates how additional contextual information affects measurable ambiguity within a fixed enterprise resource landscape. The resulting benchmark enables future studies on how retrieval systems and LLM-based agents exploit resource descriptions. Construction artefacts are provided in the appendix and repository.

2. Related Work

Constructing a benchmark for enterprise resource selection draws on four complementary strands: the retrieval methods that operate on it, the description standards and semantic enrichment whose effect it measures, the existing benchmarks whose limitations it addresses, and the ambiguity that makes its measurements meaningful. These strands motivate the design decisions underlying ORD-Bench and are surveyed in that order.

Tool and resource retrieval. LLM-based agents interact with external resources through *tool calling*: at runtime, the agent receives a natural-language request and must select and invoke the right resource from a catalogue [11, 12]. Early LLM-based agents loaded all resource descriptions statically into the prompt, but once catalogues outgrew the context window Gorilla reframed selection as an information-retrieval problem over resource descriptions [6], and three retrieval families emerged. Embedding-based ranking scores descriptions by cosine similarity and scales to large catalogues; LLM-guided pre-selection narrows the space through hierarchical filtering, as in Tool-LLM’s traversal of a category tree over 16k+ APIs [7]; and graph-based traversal exploits explicit relationships between resources for multi-hop retrieval [13]. All three are ultimately bounded by the descriptions they rank: Shi et al. report that even the strongest retrievers surface fewer than one in three of the correct tools in their top-10 across 43k real-world tools, precisely because semantically close candidates are hard to separate [8].

Resource descriptions. Retrieval quality is decided by the resource description itself, and how resources are described, made discoverable, and selectable is a problem that has outlived several generations of computing developments. Service-oriented architectures (SOA) laid the conceptual foundation: the publish–find–bind paradigm, in which providers register descriptions to a registry, consumers query it, and bind at runtime [14, 15]. One concern ran unresolved through the whole stack, from WSDL and UDDI through later REST-based formats, and resource descriptions stayed syntactic, specifying how to invoke a service rather than whether it fits a business goal.

Semantic Web Services tried to close this gap: by adding machine-interpretable meaning through shared ontologies, agents could autonomously discover, invoke, and compose services automatically rather than relying on a developer to wire them by hand [16, 17]. Hepp identified the decisive bottleneck as the manual authoring overhead of maintaining ontologies in landscapes under decentralised ownership [18]. Lightweight syntactic formats such as OpenAPI there-

fore achieved the adoption ontologies never reached, but they still captured only how an endpoint is called [19–21].

The LLM era attempts to close this semantic gap from the other side: rather than encoding meaning in formal ontologies, it relies on the reasoning capabilities of LLMs. Agent-oriented interfaces such as MCP and A2A follow this route, pairing syntactic invocation details with semantic metadata in the form of free-text capability descriptions, explicitly intended to help LLMs select the right tool at runtime [22]. But free text merely relocates the problem: Hasan et al. document systematic “description smells” in real MCP servers that measurably distort agent decisions [23]. The gap therefore persists: an agent must know not only how to call a resource, but why and when it is the right one, and neither formal ontologies nor unconstrained free text delivers this at enterprise scale. What is needed instead is a description enriched with structured, machine-interpretable business context.

Semantic enrichment and structured representations.

Structured field representations offer a practical path toward this business context: unlike flat descriptions, they supply explicit signals for capabilities, domains, and relationships. Tang et al. show that multi-field representations improve tool retrieval over flat descriptions [3], and Lu et al. show that LLM-generated metadata expansion achieves similar gains without model retraining [24]. What none of these measures against is the dominant alternative signal: dense sentence embeddings such as SimCSE compress a description into a single vector whose cosine is the de-facto similarity measure for retrieval [25]. These studies evaluate whether enrichment improves downstream retrieval, but do not examine how enrichment changes the semantic structure of a landscape, that is, whether resources become easier or harder to tell apart, and whether the typed-field view and the embedding view even agree on the answer. Realising such enrichment in practice, and studying its effect, requires a description standard that is at once *typed*, so structured fields are addressable, and *extensible*, so semantic content can be added without breaking the schema.

Open Resource Discovery. ORD [4] is exactly such a standard, and it is the description standard this benchmark builds on. Governed under Linux Foundation Europe as part of the Apeiro Reference Architecture [26], it carries institutional backing beyond a single vendor. It sits apart from adjacent industry efforts: unlike the analytics-focused Open Semantic Interchange [27] or Google’s cross-organisational Agentic Resource Discovery [5], ORD operates at the intra-enterprise level across the full breadth of enterprise assets, APIs, agents, and data products.

It provides exactly the two properties the previous strand identified as necessary. Unlike interface-centric formats, it gives each asset a typed representation with explicit fields for entity types, lines of business, capabilities, or cross-resource links. Crucially, it is *self-exposing*: system owners publish a standardised JSON document at their own endpoint, and an orchestrator harvests it without manual curation, making metadata available at the source rather

Table 1: Comparison of ORD-Bench with related tool-retrieval benchmarks. ✓ = present, × = absent. Test-case counts follow each paper’s reported total; “–” where no count is stated.

	ORD-Bench	APIBench	ToolBench	EEG	ToolLinkOS	ToolSandbox	ToolBank	ToolRet
		[6]	[7]	[28]	[29]	[9]	[30]	[8]
Resources	273	1,645	16,464	177	573	34	3,168	43,000
Test cases	350	–	126,486	503	1,569	1,032	163,000	7,600
Typed description layer	✓	×	×	×	×	×	×	×
Process models	✓	×	×	×	×	×	×	×
Tool / resource dependencies	✓	×	×	✓	✓	✓	×	×
Enrichment ablation	✓	×	×	×	×	×	×	×
Adversarial construction	✓	×	×	✓	×	×	×	×
Out-of-scope / refusal	✓	×	×	×	×	×	×	✓
Enterprise business tools	✓	×	×	✓	×	×	×	×

than requiring a central registry. Its typed fields make resource similarity explicit and therefore measurable, and its extensible schema lets semantic content be attached to a resource without altering it. Appendix A gives the ORD protocol in detail, its place within the Apeiro architecture, and a representative ORD document. Whether this typed structure and its enrichment actually improve resource distinguishability is an empirical question, and answering it requires a benchmark built for the purpose.

Benchmarks for tool and resource retrieval. Given a description standard and retrieval methods to evaluate, the question is what benchmark substrate to build on. A growing body of benchmarks evaluates whether agents can identify the appropriate tool from candidate descriptions. Scale-oriented benchmarks such as APIBench [6], ToolBench [7], and ToolRet [8] raise difficulty primarily through catalogue size. APIBank evaluates the full tool-use loop of retrieval, decision, and invocation over a hand-built API pool [31]. ToolSandbox targets stateful, multi-turn interaction rather than semantic similarity between candidates [9]. ToolBank studies tool representation learning [30], while enterprise-oriented benchmarks such as EEG [28] and ToolLinkOS [29] model relationships between tools. Table 1 compares these benchmarks with ORD-Bench.

In these benchmarks difficulty emerges implicitly from dataset composition. Still a large catalogue can be easy if descriptions are distinct, whereas a smaller one can be hard if resources share overlapping business meanings. None ties difficulty to a quantified similarity target, and none provides typed resources in paired enriched and non-enriched states. Measuring the effect of enrichment therefore requires a benchmark that controls for it explicitly.

Retrieval difficulty and adversarial construction. Whether a benchmark measures anything depends on how difficult its cases are, and in retrieval, difficulty is governed by the negatives surrounding each target. Dense passage retrieval established that training and evaluating against *hard negatives*, distractors semantically close to the answer rather than random ones, is what separates competent retrievers from trivial ones [32], and ANCE showed that periodically resampling the globally hardest negatives further sharpens this distinction [33]. At the evaluation level, BEIR showed

that retrieval accuracy collapses precisely on datasets whose candidates are topically dense [34], and MTEB generalised such difficulty-aware evaluation across dozens of embedding tasks [35]. Yet in all of these, hard negatives are mined post hoc from existing corpora; difficulty is observed, not designed.

ORD-Bench inverts this: rather than sampling confusable pairs from a fixed set, it *constructs* them, using an explicit field-based ambiguity metric to place graded distractors around every target so difficulty is a controlled quantity rather than an artefact of corpus composition. Its landscape and cases are themselves LLM-generated and LLM-adjudicated, an approach whose reliability is supported by work establishing strong LLM–human agreement in judgement tasks [36], while every generated artefact additionally passes a deterministic structural gate.

Research gap. Taken together, these strands leave a gap: no existing benchmark combines: (1) typed enterprise resource descriptions; (2) realistic resources spanning multiple business domains; (3) an explicit ambiguity metric and adversarial construction process that make semantic overlap a controlled property; and (4) paired resource states, holding the resources fixed while adding process-derived semantic enrichment, so its effect can be isolated. ORD-Bench is built to satisfy all four, serving as a foundation for future work on ambiguity analysis and retrieval over typed enterprise resources.

3. Methodology

This section describes how each of the four ORD-Bench contributions is constructed. The construction follows the Design Science Research paradigm of Österle et al. [10], iterating between design and evaluation across several build cycles. It proceeds in four steps that map directly onto the four contributions: (1) the *resource model* defines the ORD schema; (2) the *ambiguity metric and landscape generation loop* operationalise controlled difficulty; (3) the *process model and enrichment* generates semantic context; and (4) the *test-case pipeline* derives 350 test cases for systematic comparison across description conditions.

Resource model. The benchmark first needs a common representation every resource conforms to, so that similarity

Table 2: Ambiguity metric: six equal-weight dimensions plus two structural adjustments.

#	Dimension	Method and rationale
1	Text	TF-IDF cosine over title + shortDescription + description.
2	Business objects	IDF-weighted Jaccard over entity-type sets; strongest structural signal.
3	Domain	Jaccard over lineOfBusiness.
4	Keywords	Jaccard over tags.
5	Industry	Jaccard over industry.
6	Local ID	CamelCase-tokenised Jaccard over the ordId name segment.
+	Cross-ns bonus	+0.5 to numerator for cross-namespace pairs (agent sprawl).
×	Type penalty	×0.5 if resource types differ.

Formula: raw = (text + lid + ET + LoB + tags + ind + 0.5_{x-ns})/6; final = raw × 0.5 if type(A) ≠ type(B).

can be measured over the same fields and enrichment added without breaking the schema. Each resource therefore conforms to ORD v1.16 and belongs to one of three resource types (apiResource, agent, or dataProduct). Every resource contains a common set of base fields (ordId, title, shortDescription, description, lineOfBusiness, tags, and industry) together with a type-specific reference to business objects through a shared entity-type vocabulary.

Building on ORD’s extensible schema, ORD-Bench augments selected resources with semantic context by populating optional fields (e.g., capabilities) and introducing benchmark-specific extensions (useCases, partOfGroups, and processNext). This produces paired Clean-ORD and Enriched-ORD representations of the same resources, enabling controlled evaluation of semantic enrichment. Appendix B.1 provides the complete schema used in ORD-Bench, while Appendix B.2 illustrates a representative resource in both representations.

Systems and vocabulary. ORD follows a self-describing model in which each system exposes only its own resources. As identified in the related work, no existing benchmark provides such an ORD-based enterprise resource landscape, motivating the construction of a synthetic reference landscape. ORD-Bench therefore models a heterogeneous enterprise landscape comprising 10 systems from five vendors (SAP, Workday, Siemens, and generic MES and ITSM systems), covering the business domains most commonly encountered in enterprise scenarios.

To ensure semantic interoperability across systems, all resources share a common vocabulary of 50 business-object types derived from the SAP One Domain Model (sap.odm). This shared vocabulary intentionally creates cross-system ambiguity: for example, a Workday resource referencing business-object Employee is semantically related to an SAP SuccessFactors resource exposing the same entity type, making cross-namespace resource selection a retrieval challenge rather than a trivial namespace lookup. Appendix B.3 lists all 50 entity types.

Ambiguity metric. Without explicitly controlling resource

Table 3: Similarity tier examples: one representative pair (A/B) per tier. ET = entity types. The score sums text (TF-IDF), lid (localId), ET, LoB, tags, and a +0.5 cross-namespace bonus.

Resource	System
HIGH — sim = 0.74 text .44 · lid .50 · ET 1.0 · LoB 1.0 · tags 1.0 · +.5	
A Campaign Performance Analytics	emarsys.cx
ET: Campaign, CustomerSegment, CustomerAccount	
B Campaign Segmentation and ROI Analytics	sap.crm
ET: Campaign, CustomerAccount, CustomerSegment	
MEDIUM — sim = 0.50 text .38 · lid .11 · ET 1.0 · LoB 1.0 · tags 0 · +.5	
A Sales Team Availability and Capacity Service	sap.crm
ET: Employee, TimeOff	
B Employee Absence and Resource Utilization API	workday.hcm
ET: Employee, TimeOff	
LOW — sim = 0.25 text .31 · lid .17 · ET .32 · LoB 0 · tags .20 · +.5	
A Procurement Vendor Management API	corp.itsm
ET: PurchaseOrder, Vendor	
B Material and Vendor Master Data API	my.mes
ET: Material, Vendor	

similarity, a generated landscape would predominantly consist of unrelated resources, making resource retrieval trivially easy and limiting the benchmark’s usefulness. To address this, ORD-Bench introduces an ambiguity metric that quantifies how confusable two resources are based on their ORD descriptions, enabling the resource generation pipeline to control similarity and place graded distractors around every resource. The metric quantifies the ambiguity between two resources using six equally weighted dimensions plus two structural adjustments (Tab. 2). Equal weights are used by design, as no labelled confusion dataset for ORD resources exists.

The resulting ambiguity score, a number between 0 and 1, groups resource pairs into three similarity tiers. The boundaries, HIGH (≥ 0.50), MEDIUM (0.25–0.49), LOW (0.10–0.24), are derived from pilot analysis: a score above 0.50 corresponds to pairs sharing entity types, line of business, and tags simultaneously, requiring system-context to distinguish; 0.25–0.49 covers partial-field matches requiring domain context; and 0.10–0.24 captures resources with one or two shared fields. Table 3 shows one validated pair per tier; Appendix C.1 gives the full threshold rationale and further examples.

This approach is used rather than relying on embedding similarity, because ambiguity can be defined directly over interpretable ORD fields to ensure that benchmark difficulty remains transparent, reproducible, and independent of any particular embedding model.

Landscape generation. With the resource model, system landscape, and ambiguity metric defined, the landscape can be populated with resources. The goal is a landscape dense enough that no retrieval method succeeds by trivial matching. The generation follows three sequential steps (Fig. 1(a)): seed generation, iterative gap-filling, and quality control.

Seed generation starts from 30 LLM-generated seed resources (3 per system), with entity types assigned so that

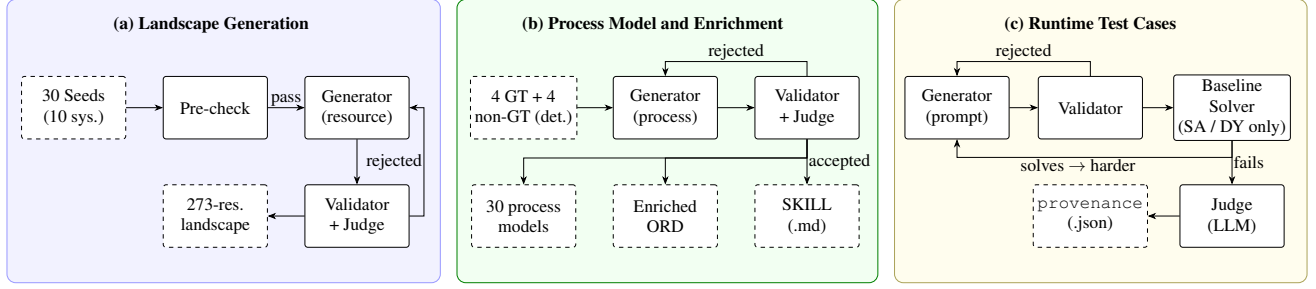


Figure 1: Benchmark construction pipelines. (a) Landscape: seeds, expanded via pre-check, LLM generation, and Validator/Judge gate. (b) Process model and enrichment: one accepted model yields Enriched-ORD fields, a SKILL.md, and design-time activity cases. (c) Runtime cases: the single-LLM baseline-solver difficulty gate applies only to the retrieval-driven modes (Skill-Adjusted, Dynamic); Skill-Guided is scored by routing alone and Out-of-Scope uses a programmatic absence check, so both bypass the gate.

all 50 `sap.odm` types are referenced from the start. From the seeds, iterative gap-filling grows the landscape round by round, generating resources at specific similarity tiers: for each existing resource, the loop aims to surround it with a neighbourhood of at least 3 HIGH-ambiguity (≥ 0.50), 5 MEDIUM-ambiguity (0.25–0.49), and 5 LOW-ambiguity (0.10–0.24) neighbours, generating a new resource whenever a tier is under-filled. Before each LLM call, a pre-check (Fig. 1(a)) verifies that a resource with the intended properties can reach the target tier; if not, the gap is skipped.

Every generated candidate passes a two-stage quality gate: a deterministic ORD v1.16 validator runs first at zero LLM cost (checking required fields), followed by an LLM Judge that evaluates coherence with the system domain, non-duplication, entity-type references, and a domain-sensible description. All LLM calls use `claude-haiku-4.5` at temperature 0.0 with a fixed seed. Accepted resources become seeds for subsequent rounds. A final pass removes near-textual-duplicates that the structural generation score could not detect. Appendix C.2 details the algorithm.

Generation targets 300 resources, a mid-scale size, large enough to create genuine cross-system confusion without forcing resources from entity-type categories that rarely appear across real enterprise systems.

Process models and semantic enrichment. After fixing the enterprise resource landscape, 30 enterprise process models (15 BPMN, 15 CMMN) are generated as controlled sources of process semantics (Fig. 1(b)). Each model is constructed from a deterministically selected resource pool consisting of four resources that have reached the required neighbourhood ambiguity level and four resources that have not, while spanning at least three namespaces. The generator creates a coherent 8-activity process model grounded in this pool, assigning one activity to each selected resource.

Each generated process model passes a two-stage validation gate. First, a deterministic validator verifies structural correctness and ensures that all referenced `ordIds` exist in the resource landscape. Second, an LLM Judge evaluates whether the generated process is semantically coherent from a domain perspective. The corresponding generator and judge prompts are provided in Appendix D.1.

Only accepted process models are used for semantic

enrichment. For each confirmed activity–resource pair, the generation pipeline writes four typed semantic fields, capabilities, useCases, partOfGroups, and processNext, back into the corresponding ORD document, creating the Enriched-ORD state. Figure 2 illustrates how process activities are mapped to ORD resources and how additional semantic context is derived without modifying resource identities or existing base fields. Furthermore, each accepted process model produces a *skill artefact*: a structured markdown representation of the process steps, with each step explicitly linked to its confirmed resource [37]. This representation makes the generated process knowledge directly consumable by LLM-based agents.

To quantify the effect of this enrichment on resource distinguishability, the ambiguity metric is extended by three of the newly introduced fields (capabilities, partOfGroups, and useCases). Each field contributes only when both resources in a comparison pair contain the respective attribute, extending the original six-field metric (Table 2) to a maximum of nine dimensions. The field processNext is excluded because it represents directed process relationships between resources rather than an intrinsic property of an individual resource. Comparing identical resource pairs under the original and extended metrics isolates the contribution of semantic enrichment to ambiguity reduction. The complete nine-dimension formulation and an illustrative example are provided in Appendix E.2.

Test cases. With the landscape and its enrichment in place, the benchmark needs concrete tasks that exercise resource selection against the graded distractors built into it. To this end it provides two case families. *Design-time cases* test activity-to-resource matching: given a process step, which landscape resource realises it? Each of the 30 process models yields eight activity cases, one per resource in the pool. Ground-truth pairings are fixed by construction: the Generator assigns one resource per activity and the two-stage gate verifies the pairing is structurally valid and domain-coherent, so every label is machine-verified rather than hand-annotated. These cases make the benchmark a substrate for automating semantic enrichment: a retrieval method that can map process activities to ORD resources can populate the enrichment fields automatically, turning process models into an annotation pipeline.

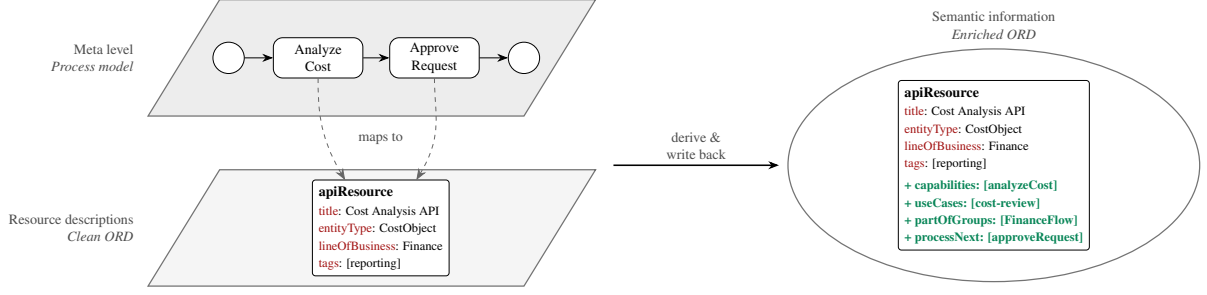


Figure 2: Process-driven semantic enrichment of ORD. A process model is mapped onto a Clean-ORD resource; the mapping deterministically derives four typed fields (*capabilities*, *useCases*, *partOfGroups*, *processNext*) written back into the resource, yielding its Enriched-ORD state. The resource identity is preserved.

Runtime cases test resource selection under real-world intent conditions. Each case targets a resource that has reached the required neighbourhood, ensuring every ground truth has genuine HIGH-tier distractors in the full landscape. Like the landscape, each case is produced by a Generator–Judge pipeline (Fig. 1(c)): a Generator produces a natural-language request targeting a specific resource, and the Judge verifies the request is unambiguous and domain-coherent. Three orchestration modes reflect the degree to which a prior process definition is available, covering the enterprise spectrum from fully-guided skill execution to open-ended retrieval: *Skill-Guided* (30 test cases) where a matching skill covers the full request; *Skill-Adjusted* (20 test cases) where a skill applies but has gap steps that require retrieval; and *Dynamic* (40 test cases) where no skill matches and the agent must select from the full landscape. An Out-of-Scope condition (20 test cases) tests abstention when no resource can satisfy the request.

For the retrieval-driven modes, a Baseline-Solver gate additionally discards any case a trivial single-call LLM solver already solves, certifying that retained cases require structured retrieval; *Skill-Guided* and *Out-of-Scope* bypass this gate by construction. The test cases themselves are fixed; every case can run on both the Clean-ORD and Enriched-ORD landscape states, so the enrichment effect is measurable as a controlled variable without changing the request or the ground truth. Appendix D.2 gives the per-mode generator prompts and a solver-gate provenance trace.

4. Results

This section reports the artefacts produced by applying the methodology of Sec. 3. The resource landscape is presented first, followed by the test-case suite derived on top of it. Finally, an ambiguity analysis compares resource ambiguity under clean and semantically enriched description.

Resource Landscape. The generation loop began at round 0 with 30 seed resources (3 per system) and ran for 13 further rounds (Fig. 3). Acceptances fell steadily as the landscape filled: round 1 accepted 152 of 465 candidates, round 3 only 48, and by round 13 just one candidate passed the gate. The falling acceptance count and the flattening HIGH-tier curve are two views of the same saturation, as more resources were placed, new candidates had to be simi-

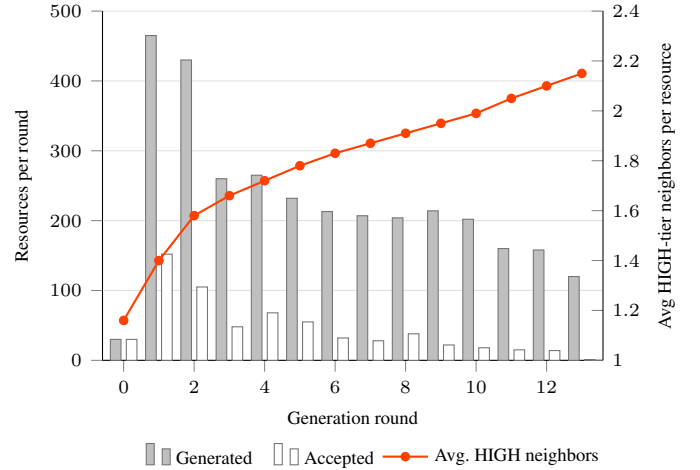


Figure 3: Iterative generation convergence. Bars: candidates generated (grey) and accepted (white) per round. Circles: avg. HIGH-tier neighbours ($\text{sim} \geq 0.50$) under the generation-time similarity (no TF-IDF text, for speed).

lar enough to fill remaining neighbourhood gaps yet distinct enough to clear the quality gate, a window that narrowed with every round. The average HIGH-tier neighbourhood accordingly rose steeply at first (1.16 at round 0 to 1.72 by round 4) and then flattened towards 2.15, since each newly added resource contributed fewer novel HIGH-tier pairings once the dense regions were already covered.

Generation stopped at the 300-resource target. By that point acceptances had collapsed to one per round, so the loop had naturally saturated. A final near-duplicate pass recomputed full similarity on the fixed 300-resource corpus, including the TF-IDF text score omitted during generation for speed, and removed 27 near-duplicate pairs, leaving 273 resources.

The quality gate filtered most generated candidates. Of those that reached the LLM Judge, 2,936 were rejected. The most common reasons (which can overlap for a single candidate) were domain incoherence (2,363 cases, for example an IT-service-management ticket proposed inside an ERP finance system), unjustified entity-type references (1,720), and duplication of an existing resource (1,393). A further 85 were discarded by the pre-check before any LLM call. An example rejection log entry is in Appendix C.2.

The resulting landscape (Table 4) spans ten systems whose

Table 4: ORD-Bench landscape: 10 namespaces, 273 resources. GT = resources that satisfy the required neighbourhood ($\text{HIGH} \geq 3$, $\text{MEDIUM} \geq 5$, $\text{LOW} \geq 5$).

Namespace	Domain	API	Agt	DP	Tot	GT
sap.s4	ERP, Finance	14	6	8	28	10
sap.sf	HR	15	6	5	26	4
sap.ariba	Procurement	13	3	7	23	7
sap.crm	Sales, Customer	11	7	9	27	7
sap.ehs	Safety, Env.	15	6	9	30	4
corp.itsm	IT Service Mgmt	14	8	7	29	9
my.mes	Mfg. Execution	15	5	9	29	9
workday.hcm	HR (non-SAP)	13	9	4	26	6
emarsys.cx	Marketing, CX	13	8	8	29	8
siemens.plm	Prod. Lifecycle	14	8	4	26	6
Total		137	66	70	273	70

resources cluster around shared ODM entity types, producing the confusable cross-system neighbourhoods the benchmark targets. Appendix E.1 provides a graph visualisation that makes this cross-system similarity visible.

Recomputed on the final 273-resource corpus under the full similarity metric (including the TF-IDF text score), each resource carried on average 1.74 HIGH-tier, 11.0 MEDIUM-tier, and 49.8 LOW-tier neighbours; this figure is lower than the 2.15 reported during generation (Fig. 3), because the full metric is stricter than the fast generation-time score and the near-duplicate pass removed 27 resources. Exactly 70 of the 273 resources reached the required neighbourhood defined in Sec. 3 ($\text{HIGH} \geq 3$, $\text{MEDIUM} \geq 5$, $\text{LOW} \geq 5$), forming the ground-truth-eligible set used as targets for retrieval test cases. The resource-type distribution (50% `apiResource`, 24% `agent`, 26% `dataProduct`) mirrors real enterprise landscapes. The 70 eligible resources were not distributed uniformly: `sap.ehs` and `sap.sf` contributed fewer because their entity types (safety incidents, HR objects) are less cross-domain than those of `corp.itsm` or `my.mes`, which share business objects (Vendor, Material) with several systems.

Semantic Enrichment. The semantic enrichment is derived from generated enterprise process models. Of 57 process-model generation attempts, 30 were accepted (8 rejected by the deterministic validator, 16 by the LLM Judge, and 3 abandoned after the retry budget). Each accepted model uses four resources that have reached the required ambiguity neighbourhood and four that have not. Only the former receive the four process-derived semantic fields; the supporting resources stay at Clean-ORD level. Enrichment is therefore targeted at the 70 confusable resources rather than landscape-wide; Appendix B.2 shows a representative resource in its Clean-ORD and Enriched-ORD state.

To test whether this semantic enrichment improves distinguishability between resources, the six-field and the extended ambiguity metric are compared on the same resource pairs. Figure 4 shows the effect across all 2,415 pairs. Across all $\binom{70}{2} = 2,415$ pairs, the mean pairwise ambiguity dropped from 0.111 to 0.084 (−24.5%). Looking only at the 85 pairs whose six-field score exceeded the HIGH threshold (≥ 0.50), the ones that actually risk confusion, the

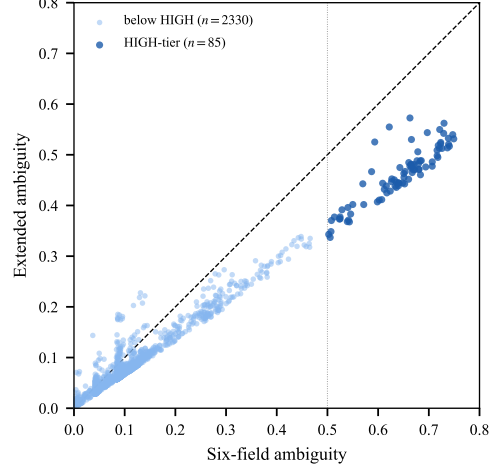


Figure 4: Disambiguation effect of the semantic enrichment. Each point is one of the 2 415 pairs among the 70 enriched resources: six-field ambiguity (x) versus extended nine-dimension ambiguity (y). All 85 HIGH-tier pairs (≥ 0.50 , dark) drop below the diagonal, and drop most where original ambiguity was highest. Low-ambiguity pairs (light) mostly stay near the diagonal; a minority rise slightly. Overall, 2 176 of the 2 415 pairs move apart.

mean dropped from 0.643 to 0.461 (−28.3%).

Enrichment lowered structural ambiguity most where the risk of confusion was highest. The effect is also visible at the resource level: 52 of the 273 resources shed at least one HIGH-tier neighbour under the extended metric while none gained one, dissolving 128 HIGH-tier pairings in total. Enrichment therefore does not merely shift averages but pulls confusable resources apart into distinguishable ones. An embedding-based view of the same enrichment, which moves in the opposite direction, is analysed in Sec. 5.

Test-Case Suite. On top of the landscape the benchmark ships 350 test cases as a reusable substrate, split into a *design-time set* and a *runtime set*. The design-time set reuses the 30 accepted process models: each contributes eight activity cases, giving 240 design-time cases with four activities drawn from resources that had reached the required neighbourhood and four from those that had not.

The *runtime set* spans four modes, each with its own acceptance criterion. The two retrieval-driven modes, Skill-Adjusted and Dynamic, were built under a Baseline-Solver gate that regenerated a case until a trivial single-call solver could no longer answer it: across their 60 cases this took 118 generation iterations, 37 rejected as too easy and regenerated with more implicit phrasing, and a further 25 rejected by the Judge for realism or naming violations. Skill-Guided (30 cases) is scored by routing accuracy rather than retrieval difficulty, so it bypasses the solver gate; all 30 were accepted on the first attempt. Out-of-Scope (20 cases) has no ground-truth resource, so instead of the solver gate an absence check discards any request that lies too close to an existing resource; this rejected 9 of the 29 generated candidates. Together the four modes yield 110 runtime cases, bringing the benchmark to 350 test cases.

Table 5: One test case per family. DT = design-time, SG = Skill-Guided, SA = Skill-Adjusted, DY = Dynamic, OOS = Out-of-Scope.

Family	<i>n</i>	Example request / step label	Ground-truth target
DT	240	<i>“Request Production Schedule Adjustment — initiate a change request to modify production orders based on demand forecasting or supply constraints.”</i>	sap.crm:apiResource: MachineProductionOrderIntegration
SG	30	<i>“We need to handle a product engineering change end to end — from assessing production impact through validating materials are compliant with the new design.”</i>	Routes to skill proc_022 (4 confirmed steps)
SA	20	<i>“Establish a procurement and delivery workflow coordinating vendor assessment and enriched customer purchases, including handling customer grievances and reconciling disagreements.”</i>	proc_033 + gaps: CustomerOrderFulfillmentTracking, CustomerAccountDisputeManagement
DY	40	<i>“Pull up a recent issue a customer reported last week so I can see what we promised and whether we’re on track.”</i>	corp.itsm:apiResource: ServiceTicketOrderManagementAPI
OOS	20	<i>“Automatically flag suppliers whose credit ratings deteriorated in the past 90 days so we can reduce order volumes before a cash-flow problem.”</i>	None — correct action is refusal

To confirm that difficulty was stable rather than an artefact of a single generation-time solver run, the 90 retrieval-driven runtime cases were re-certified by running the baseline solver five independent times per case and flagging any case it solved. Under the strict criterion (solved in zero of five runs), 91.1% of cases (82/90) certified as robust, with a mean difficulty of 0.94; per mode, Skill-Guided certified 30/30, Skill-Adjusted 18/20, and Dynamic 34/40, with the six residual cases solved on only a minority of runs. Difficulty was therefore a reproducible property of the cases rather than a single-run coincidence.

Table 5 shows one accepted case per family. Design-time cases were terse activity labels resolved against the in-model resource pool; runtime cases were full natural-language requests resolved against the whole landscape. Each mode added a distinct source of difficulty: typed distractors, implied gap steps, cross-namespace confusables, or a genuinely absent capability. How retrieval methods perform on these cases is the subject of a companion paper.

5. Discussion

This section first interprets the results with respect to the research question, before discussing the limitations of the work and directions for future research.

Interpretation of findings. The benchmark fills a specific gap: its difficulty is a controlled property of semantic ambiguity between typed enterprise resources, not a by-product of catalogue size. Three properties make its results transparent and traceable: (1) the auditable ambiguity metric assigns every pair a tier that can be traced back to the six ORD fields; (2) the adversarial difficulty gate removes trivial test cases, so measured accuracies reflect genuine selection ability rather than easy wins; (3) the clean and enriched description states isolate the semantic enrichment Δ as an independent factor, so an observed gain is attributable to the semantic fields and not to case-difficulty drift. The five-run difficulty certification (91.1% of retrieval-driven cases robust) confirms that these properties hold in practice: the benchmark is not merely designed to be hard, but demonstrably is.

Field-based versus embedding difficulty. Scoring difficulty from typed fields rather than embeddings serves two

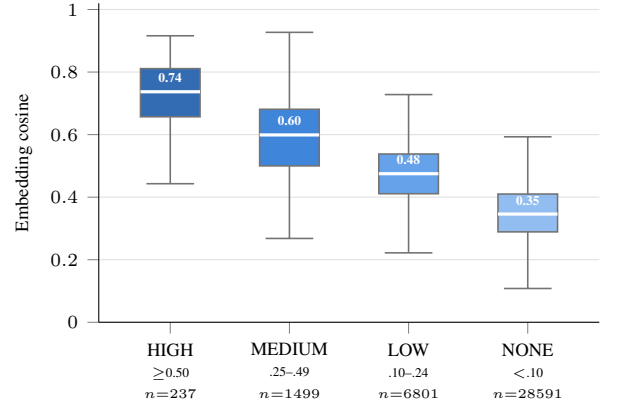


Figure 5: Embedding cosine grouped by structural tier (37 128 pairs). Median cosine rises monotonically with the tier (NONE → HIGH), confirming the field-based metric captures a real difficulty signal independent of embeddings ($r=0.63$ with structural score).

purposes: it keeps every tier assignment auditable, and it avoids handing embedding-based retrievers a difficulty structure defined in their own representation, which keeps the benchmark fair across retrieval families.

Furthermore, the metric captures a real difficulty signal rather than an arbitrary construct. This is confirmed by two findings: First, the field-based tiers and an independent embedding model (text-embedding-3-large) agree on which pairs are confusable: their scores correlate at $r = 0.63$, and median embedding cosine rises monotonically with the structural tier, from 0.35 for unrelated pairs to 0.74 for HIGH-tier pairs (Fig. 5). The structural metric is therefore not an arbitrary construct; it captures a difficulty signal a completely different representation also sees. Second, a PCA projection of the same embeddings (Appendix E.1) shows the ten namespaces fully intermixed while resource types form distinct regions, as resources group by what they do, not by which system they belong to, matching the metric’s cross-type penalty. At the same time the agreement is only partial ($r = 0.63$, not 1.0), and this gap is intentional: the metric adds a cross-type penalty and a cross-namespace bonus that raw embedding cosine does not have. Benchmark difficulty is thus deliberately *not* a copy of what an embedding retriever measures.

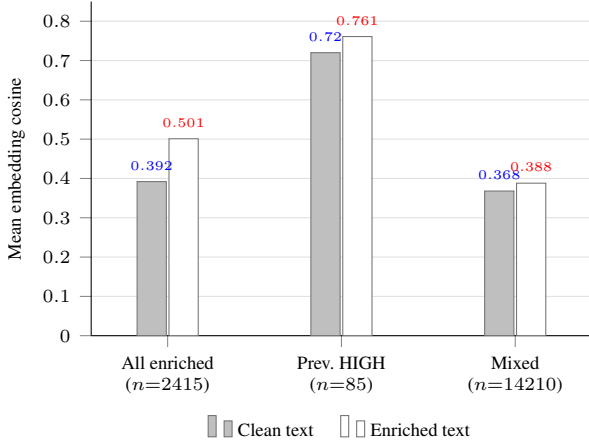


Figure 6: Embedding cosine on the same pairs, clean text (grey) versus enriched text (white). Unlike the structural metric in Fig. 4, embedding cosine *rises* in every group once semantic text is added. The mixed group ($n=14\,210$) covers pairs where only one resource is enriched.

Influence of semantic enrichment. Comparing the same resources under clean and enriched descriptions produces the paper’s sharpest lesson. Writing the process-derived semantic fields back into a resource moves the two similarity measures in *opposite* directions on the very same pairs. The structural metric drops by up to 28.3% on the most confusable pairs (Fig. 4), whereas embedding cosine *rises*: with the clean text embedding `title`, `shortDescription` and `description`, and the enriched text appending the four semantic fields, the mean cosine across all 2,415 both-enriched pairs climbs from 0.392 to 0.501 (+27.7%), and it rises even for mixed pairs where only one resource is enriched (Fig. 6).

This is not a contradiction but a statement about representation. A method that reads the typed fields as discrete signals (namespace, entity-type, capability filters) sees two resources with disjoint capabilities pushed apart and disambiguates better; a method that compresses every field into one vector sees the added topical text pull the vectors together and disambiguates worse. Enrichment thus helps only when the retrieval mechanism and the enriched field types are structurally aligned. A benchmark that fixed a single description state could not expose this dependency at all, which is precisely why ORD-Bench ships both states.

Asymmetric enrichment. One design choice affects how the enrichment results should be read: the Enriched-ORD state writes semantic fields only into the 70 highly ambiguous resources, not into the 203 non-eligible distractors. For methods reading typed fields, this asymmetry is intentional: it widens the distinctive gap between the target resource and its competitors, exactly what the enrichment Δ measures. For embedding-based retrieval it can work in the opposite direction: the target resource’s vector shifts into a richer topical region while un-enriched neighbours stay put, which in some configurations reduces cosine distance to resources in adjacent topics. The mixed-pair experiment ($n=14,210$ pairs where only one resource is enriched) directly tests

this: mean embedding cosine rises there too, from 0.368 to 0.388 (+5.3%), with 10,517 of the 14,210 pairs becoming more similar, confirming the vector-blurring mechanism is not an artefact of the 70-resource asymmetry but a property of how embeddings respond to added context regardless of which side of the pair is enriched.

Limitations. A first group of limitations concerns the ambiguity metric itself. Equal weights across the six dimensions are a deliberate, auditable default, since differential weights would require a labelled confusion dataset that does not exist for ORD resources. A sensitivity analysis varying the weights is the natural next step, and the enrichment effect is only robust if it persists under plausible re-weightings. A related caveat concerns the embedding-cosine result: enrichment adds typed fields but leaves the base free-text descriptions (`description`, `shortDescription`) unchanged, so the observed rise in cosine may partly reflect appended structured tokens rather than genuine semantic convergence. Fairly evaluating embedding methods under *full* enrichment would require rewriting the free-text descriptions too, which the current design holds fixed to keep the two states comparable.

A second group concerns test case ground truth and difficulty. Ground truth is fixed by construction and machine-verified (deterministic validator plus LLM Judge) rather than confirmed by independent human annotators, and no held-out partition was constructed. Both a human-validated gold set and a held-out split are future priorities. Difficulty was certified over five solver runs, but the solver is not fully deterministic, so a few cases were solved on only one or two runs and sit close to the difficulty cut-off; more runs would place them more reliably.

The suite is also intentionally modest in size: 350 cases split into a design-time set and four runtime families (Skill-Guided, Skill-Adjusted, Dynamic, Out-of-Scope). Because the cases are adversarially constructed rather than randomly sampled, the design prioritises *systematic coverage* of the retrieval scenarios over statistical representativeness: each family targets a distinct selection challenge (typed distractors, implied gap steps, cross-namespace confusables, and genuinely absent capabilities), and each case is a purpose-built ambiguity configuration surrounded by graded distractors. The benchmark therefore emphasises representative coverage of enterprise selection scenarios rather than sample size. A consequence is that per-family case counts are small, so any downstream evaluation on individual families should report results as directional rather than as statistically powered estimates; scaling each family while preserving its controlled construction is left to future work.

A final group concerns scale and realism. The landscape (273 resources) is intentionally mid-scale, large enough to make retrieval non-trivial but far below industrial catalogues, because the benchmark targets semantic selection under ambiguity, not scale. Its size is in fact bounded by the ten-system design: with a fixed set of namespaces and entity types the loop saturates once the dense regions are covered, so scaling further is primarily a matter of adding

systems and entity-type vocabularies rather than generating more resources within the existing ones. The benchmark also stops at resource *selection*: whether a selected resource executes a process step successfully is not measured.

Finally, resources are LLM-generated rather than harvested from production systems; while every resource passes a structural and semantic gate, real ORD documents may exhibit description smells the benchmark does not reproduce [23], which motivates a productive endpoint extension.

Future work. Natural extensions are scaling to 10^3 – 10^4 resources, adding a held-out partition with multi-annotator ground truth, widening the certification run budget to sharpen the difficulty cut-off around the residual borderline cases, harvesting a subset from productive ORD endpoints to complement the synthetic landscape, extending the number of test cases, and substrate beyond selection to end-to-end task completion.

Furthermore, current enrichment is derived solely from process models. Other enterprise architecture artefacts such as organisational structures, data models, or system documentation could serve as complementary enrichment sources, broadening the semantic context written back into ORD. On the metric side, beyond varying the field weights, a dropout-style ablation that removes one enrichment field at a time would isolate which fields carry disambiguating signals, and a hybrid metric combining typed-field signals with embedding similarity for the free-text dimension could narrow the intentional gap between the two representations. Released as an open dataset, ORD-Bench would give the community a typed-description-layer counterpart to catalogue-scale benchmarks such as Gorilla/APIBench [6] and ToolBench [7].

6. Conclusion

This paper introduced ORD-Bench to answer the question: *How can a benchmark for enterprise resources be constructed that makes the influence of structured and semantically enriched descriptions on resource distinguishability measurable?* The four contributions address each requirement in turn. (1) A typed enterprise resource landscape of 273 resources across ten systems, represented natively in ORD, provides the structured substrate. (2) A field-based ambiguity metric combined with an adversarial construction loop generates controlled distractor density rather than relying on catalogue size alone, making difficulty an auditable property of the landscape. (3) 30 enterprise process models with derived skill artefacts augment selected resources with semantic context while leaving the resource set unchanged, so the enrichment effect can be isolated. (4) 350 test cases across both description conditions form a reusable substrate for LLM-based resource retrieval and selection studies.

Together these contributions directly answer the research question: ORD-Bench makes the influence of structured and semantically enriched descriptions on resource distinguishability measurable, and a first pairwise ambiguity analysis on the resulting artefact finds that semantic enrichment

lowers structural ambiguity yet raises embedding-based similarity on the very same resource pairs (Sec. 4). The core lesson is that an enterprise resource landscape can be described in multiple ways, and the choice of representation determines what ambiguity means and which retrieval mechanisms can exploit it. ORD-Bench is released openly as a typed, process-grounded substrate for studying this dependency.

References

- [1] W. Xu, C. Huang, S. Gao, and S. Shang, “Llm-based agents for tool learning: A survey,” *Data Sci. Eng.*, vol. 10, no. 4, pp. 533–563, 2025. [Online]. Available: <https://doi.org/10.1007/s41019-025-00296-9>
- [2] M. Dumas, F. Milani, and D. Chapela-Campa, “Agentic business process management systems,” in *Business Process Management Workshops - BPM 2025 International Workshops, Seville, Spain, August 31-September 5, 2025, Revised Selected Papers*, ser. Lecture Notes in Business Information Processing, I. van de Weerd, B. Estrada-Torres, and H. van der Aa, Eds., vol. 569. Springer, 2025, pp. 3–14. [Online]. Available: https://doi.org/10.1007/978-3-032-13426-4_1
- [3] Y. Tang, W. Su, Y. Liu, and Q. Ai, “Multi-field tool retrieval,” *CoRR*, vol. abs/2602.05366, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.05366>
- [4] Linux Foundation Europe. Open resource discovery. [Online]. Available: <https://open-resource-discovery.org/>
- [5] J. Bu and S. Krishnan. Announcing the agentic resource discovery specification—google developers blog. [Online]. Available: <https://developers.googleblog.com/announcing-the-agentic-resource-discovery-specification/>
- [6] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html
- [7] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” in *The Twelfth International Conference on Learning*

Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=dHng200Jjr>

- [8] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren, “Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models,” in *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, ser. Findings of ACL, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds., vol. ACL 2025. Association for Computational Linguistics, 2025, pp. 24 497–24 524. [Online]. Available: <https://doi.org/10.18653/v1/2025.findings-acl.1258>
- [9] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, “Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities,” in *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, ser. Findings of ACL, L. Chiruzzo, A. Ritter, and L. Wang, Eds., vol. NAACL 2025. Association for Computational Linguistics, 2025, pp. 1160–1183. [Online]. Available: <https://doi.org/10.18653/v1/2025.findings-naacl.65>
- [10] H. Österle, J. Becker, U. Frank, T. Hess, D. Karagianis, H. Krcmar, P. Loos, P. Mertens, A. Oberweis, and E. J. Sinz, “Memorandum on design-oriented information systems research,” *Eur. J. Inf. Syst.*, vol. 20, no. 1, pp. 7–10, 2011. [Online]. Available: <https://doi.org/10.1057/ejis.2010.55>
- [11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
- [12] OpenAI. Function calling and other API updates. [Online]. Available: <https://openai.com/index/function-calling-and-other-api-updates/>
- [13] F. Nizar, E. Lumer, A. Gulati, P. H. Basavaraju, and V. K. Subbiah, “Agent-as-a-graph: Knowledge graph-based tool and agent retrieval for LLM multi-agent systems,” in *Proceedings of the 18th International Conference on Agents and Artificial Intelligence, ICAART 2026 - Volume 3, Marbella, Spain, March 5-7, 2026*, A. P. Rocha, M. Wahde, and H. J. van den Herik, Eds. SCITEPRESS, 2026, pp. 1975–1984. [Online]. Available: <https://doi.org/10.5220/0014473600004052>
- [14] I. Melzer, *Service-Orientierte Architekturen Mit Web Services: Konzepte - Standards - Praxis*, 4th ed. Spektrum Akademischer Verlag GmbH.
- [15] F. Curbera, M. J. Duftler, R. Khalaf, W. Nagy, N. Mukhi, and S. Weerawarana, “Unraveling the web services web: An introduction to soap, wsdl, and UDDI,” *IEEE Internet Comput.*, vol. 6, no. 2, pp. 86–93, 2002. [Online]. Available: <https://doi.org/10.1109/4236.991449>
- [16] S. A. McIlraith, T. C. Son, and H. Zeng, “Semantic web services,” *IEEE Intell. Syst.*, vol. 16, no. 2, pp. 46–53, 2001. [Online]. Available: <https://doi.org/10.1109/5254.920599>
- [17] C. Feier, A. Polleres, D. Roman, J. Domingue, M. Stollberg, and D. Fensel, “Towards intelligent web services: The web service modeling ontology (WSMO).” [Online]. Available: <https://oro.open.ac.uk/23147/1/10.1.1.94.1336%5B1%5D.pdf>
- [18] M. Hepp and D. Roman, “An ontology framework for semantic business process management,” in *eOrganisation: Service-, Prozess-, Market-Engineering: 8. Internationale Tagung Wirtschaftsinformatik - Band 1, WI 2007, Karlsruhe, Germany, February 28 - March 2, 2007*, A. Oberweis, C. Weinhardt, H. Gimpel, A. Koschmider, V. Pankratius, and B. Schnizler, Eds. Universitätsverlag Karlsruhe, 2007, pp. 423–440. [Online]. Available: <http://aisel.aisnet.org/wi2007/27>
- [19] C. Pautasso, O. Zimmermann, and F. Leymann, “Restful web services vs. “big” web services: making the right architectural decision,” in *Proceedings of the 17th International Conference on World Wide Web, WWW 2008, Beijing, China, April 21-25, 2008*, J. Huai, R. Chen, H. Hon, Y. Liu, W. Ma, A. Tomkins, and X. Zhang, Eds. ACM, 2008, pp. 805–814. [Online]. Available: <https://doi.org/10.1145/1367497.1367606>
- [20] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Springer, 2017, pp. 195–216. [Online]. Available: https://doi.org/10.1007/978-3-319-67425-4_12
- [21] S. Schwichtenberg, C. Gerth, and G. Engels, “From open API to semantic specifications and code adapters,” in *2017 IEEE International Conference on Web Services, ICWS 2017, Honolulu, HI, USA, June 25-30, 2017*, I. Altintas and S. Chen, Eds. IEEE, 2017, pp. 484–491. [Online]. Available: <https://doi.org/10.1109/ICWS.2017.56>

- [22] Specification. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-11-25>
- [23] M. M. Hasan, H. Li, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions." [Online]. Available: <https://arxiv.org/abs/2602.14878>
- [24] X. Lu, H. Huang, R. Meng, Y. Jin, W. Zeng, and X. Shen, "Tools are under-documented: Simple document expansion boosts tool retrieval," in *The Fourteenth International Conference on Learning Representations 26*. [Online]. Available: <https://openreview.net/forum?id=g9D9MgG7iW>
- [25] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 6894–6910. [Online]. Available: <https://doi.org/10.18653/v1/2021.emnlp-main.552>
- [26] SAP SE. Apeiro reference architecture. [Online]. Available: <https://apeirora.eu/>
- [27] The Open Semantic Interchange authors. Open semantic interchange. [Online]. Available: <https://open-semantic-interchange.org/>
- [28] S. Bansal, S. S. Sistla, A. Arikatala, and S. Schreiber, "Planning agents on an ego-trip: Leveraging hybrid ego-graph ensembles for improved tool retrieval in enterprise task planning," in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, IJCNLP-AAACL 2025, Mumbai, India, December 20-24, 2025*, K. Inui, S. Sakti, H. Wang, D. F. Wong, P. Bhattacharyya, B. Banerjee, A. Ekbal, T. Chakraborty, and D. P. Singh, Eds. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics, 2025, pp. 903–918. [Online]. Available: <https://doi.org/10.18653/v1/2025.findings-ijcnlp.54>
- [29] E. Lumer, P. H. Basavaraju, M. Mason, J. A. Burke, and V. K. Subbiah, "Graph rag-tool fusion," *CoRR*, vol. abs/2502.07223, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2502.07223>
- [30] S. Moon, S. Jha, L. E. Erdogan, S. Kim, W. Lim, K. Keutzer, and A. Gholami, "Efficient and scalable estimation of tool representations in vector space," *CoRR*, vol. abs/2409.02141, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2409.02141>
- [31] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "API-bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 2023, pp. 3102–3116. [Online]. Available: <https://doi.org/10.18653/v1/2023.emnlp-main.187>
- [32] V. Karpukhin, B. Oğuz, S. Min, P. S. H. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. Association for Computational Linguistics, 2020, pp. 6769–6781. [Online]. Available: <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [33] L. Xiong, C. Xiong, Y. Li, K. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=zeFrfgYzln>
- [34] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, 2021*. [Online]. Available: <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/65b9eea6e1cc6bb9f0cd2a47751a186f-Abstract-round2.html>
- [35] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik, Croatia, May 2-6, 2023*. Association for Computational Linguistics, 2023, pp. 2014–2037. [Online]. Available: <https://doi.org/10.18653/v1/2023.eacl-main.148>
- [36] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, 2023*. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [37] Equipping agents for the real world with agent skills. [Online]. Avail-

able: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

A. Open Resource Discovery

This appendix provides the background on ORD that the benchmark builds on: its role within the broader Apeiro Reference Architecture, and the concrete mechanisms of the protocol itself. The benchmark landscape (Appendix B) is a self-contained instantiation of the resource model described here.

A.1 Apeiro Reference Architecture

ORD is one component of the Apeiro Reference Architecture contributed by SAP to the Important Project of Common European Interest on Next Generation Cloud Infrastructure and Services (IPCEI-CIS) [26]. The Apeiro stack consists of four components that together form a Data Fabric for enterprise landscapes: Open Resource Discovery (ORD) as the metadata publication protocol, the Unified Customer Landscape (UCL) as the system-level aggregator, the Unified Metadata Service (UMS) as the central metadata service, and a Knowledge Graph as the AI-enriched semantic layer on top.

Open Resource Discovery (ORD). ORD is a machine-readable metadata protocol that lets enterprise systems self-describe their exposed resources and capabilities. It does not replace existing specifications such as OpenAPI. Instead, it describes the broader context (shared high-level information, taxonomy, and cross-resource relations) and standardises how that context is published and discovered. Each participating system implements a read-only Service Provider Interface. A central aggregator crawls all providers and re-exposes the result as a connected, queryable landscape model. Beyond APIs and events, ORD covers higher-level concepts such as Entity Types (business objects), Data Products, Integration Dependencies, and Capabilities, grouped through Packages and Consumption Bundles and related to each other through typed links.

Unified Customer Landscape (UCL). The Unified Customer Landscape (UCL) is SAP's ORD Aggregator for the customer system landscape [4, 26]. It automatically discovers and crawls ORD documents from all participating system instances and consolidates them into a unified, machine-readable graph-like landscape model. The model covers SAP, partner, and on-premise application tenants together with their API, event, and connectivity metadata. The aggregated model is re-exposed through industry-standard GraphQL and OData APIs, enabling other services and tools to query the full landscape without direct integration with each individual system. UCL thereby realises the ORD Aggregator role and provides the landscape view that UMS and the Knowledge Graph build on.

Unified Metadata Service (UMS). The Unified Metadata Service (UMS) builds on the landscape model provided by UCL and exposes the aggregated ORD-based metadata as a centrally queryable service [26]. UMS makes it possible for platforms and AI agents to query resource metadata across all participating providers through a single endpoint, without requiring direct integration with each individual system. Together with ORD, UMS removes the need for point-to-point wiring between services. UMS is not yet publicly available.

Knowledge Graph. The Knowledge Graph builds on the metadata aggregated by UMS and constructs a connected network of semantic relationships across the service landscape [26]. AI-assisted modelling further enriches the graph beyond what ORD and UMS expose structurally, so that the Knowledge Graph also captures inferred semantic connections between business objects across providers. The resulting graph serves as the basis for a semantic representation tailored for consumption by agentic AI, and is used in steps such as API discovery and query generation at runtime. The Knowledge Graph thereby extends the structural foundation of ORD with a layer of AI-derived semantics. It is not yet publicly available.

A.2 ORD Protocol

This section elaborates the concrete mechanisms of the ORD protocol. The main text introduces ORD through its typed and self-exposing nature and positions it against earlier description standards, but does not detail the concrete form of an ORD entry, the System Landscape Model, or how cross-resource references stay resolvable across systems and teams.

Three roles structure the protocol. An *ORD Provider* implements the Service Provider Interface and makes its metadata discoverable. An *ORD Aggregator* crawls many providers, combines their output with central metadata sources, and re-exposes the result as a connected, queryable landscape model. An *ORD Consumer* queries the aggregator rather than individual providers, obtaining a pre-aggregated view of the landscape. One system can hold several roles simultaneously.

Concrete entry format. Listing 1 shows a representative ORD document for a Dispute Resolution use case. The document root carries the protocol version, the applicable policy levels, and the `describedSystemInstance` that anchors all contained descriptions to a specific tenant. The document then contains four resource types whose interplay illustrates the key ORD concepts: an `agent` entry that declares the AI component, an `apiResource` for the A2A endpoint, an `integrationDependency` that names an upstream service, and a `package` that groups them under a single unit.

For the agent entry, six fields carry the semantic load. The `ordId(sap.foo:agent:disputeAgent:v1)` is the globally unique identifier under which every other resource references this agent. `relatedEntityTypes` pins it to business objects, including the cross-system ODM type `sap.odm:entityType:Dispute:v1` that expresses a shared semantic reference. `exposedApiResources` links to the concrete A2A endpoint, `integrationDependencies`

Listing 1: Representative ORD document for a Dispute Resolution use case (adapted from [4, 26]). The document root anchors descriptions to a system instance; agent, API resource, integration dependency, and package entries illustrate cross-resource referencing.

```

1 {
2   "openResourceDiscovery": "1.15",
3   "policyLevels": ["sap:core:v1", "sap:ai-agent:v1"],
4   "describedSystemInstance": {
5     "baseUrl": "https://tenant1.example-sap-system.com"
6   },
7   "agents": [{
8     "ordId": "sap.foo:agent:disputeAgent:v1",
9     "title": "Dispute Resolution Agent",
10    "shortDescription":
11      "AI agent specialized in financial dispute case resolution",
12    "version": "1.0.3",
13    "partOfPackage": "sap.foo:package:ord-reference-app:v1",
14    "relatedEntityTypes": [
15      "sap.foo:entityType:DisputeCase:v1",
16      "sap.foo:entityType:DisputeResolution:v1",
17      "sap.odm:entityType:Dispute:v1" ],
18    "exposedApiResources": [
19      {"ordId": "sap.foo:apiResource:DisputeResolutionAgent:v1"}],
20    "integrationDependencies": [
21      "sap.foo:integrationDependency:DisputeCaseManagement:v1"],
22    "partOfGroups": [
23      "sap.aicore:llmModel:sap.aicore:anthropic--claude-3.7-sonnet"],
24    "partOfGroups": ["sap.eab:process:sap.foo:DisputeResolutionProcess"],
25    "labels": {"foo.ai:interaction-mode":
26      ["conversational", "system-triggered"]},
27    "tags": ["finance", "dispute-resolution", "ai-agent"]
28  }],
29  "apiResources": [{
30    "ordId": "sap.foo:apiResource:DisputeResolutionAgent:v1",
31    "title": "Dispute Resolution Agent",
32    "apiProtocol": "a2a",
33    "partOfPackage": "sap.foo:package:ord-reference-app:v1",
34    "resourceDefinitions": [{
35      "type": "a2a-agent-card",
36      "mediaType": "application/json",
37      "url": "/definitions/DisputeResolutionAgentcard.json",
38      "accessStrategies": [{"type": "open"}]
39    }],
40    "partOfConsumptionBundles": [
41      {"ordId": "sap.joule:consumptionBundle:mTLS:v1"}],
42    "exposedEntityTypes": [
43      {"ordId": "sap.foo:entityType:DisputeCase:v1"},
44      {"ordId": "sap.odm:entityType:Dispute:v1"}]
45  }],
46  "integrationDependencies": [{
47    "ordId":
48      "sap.foo:integrationDependency:DisputeCaseManagement:v1",
49    "title": "Dispute Case Management Integration",
50    "mandatory": true,
51    "aspects": [{
52      "apiResources": [
53        {"ordId": "sap.bar:apiResource:DisputeResolutionAPI:v1"}]
54      }
55    ]
56  }],
57  "packages": [{
58    "ordId": "sap.foo:package:ord-reference-app:v1",
59    "title": "ORD Reference Application Package",
60    "version": "1.0.0",
61    "vendor": "sap:vendor:SAP:"
62  }]
63 }

```

declares upstream integrations, `partOfGroups` records both the LLM model backing the agent and its business-process group membership, and `labels/tags` carry free-form taxonomy. The same pattern reappears for every ORD resource type, enabling uniform retrieval without per-resource parsing.

The `apiResource` entry adds two things the agent entry does not: `resourceDefinitions` points to the machine-readable A2A agent card, and `partOfConsumptionBundles` declares the authentication bundle (`sap.joule:consumptionBundle:mTLS:v1`) through which consumers may call the endpoint. The `integrationDependency` entry then references an API published by yet another system (`sap.bar`), showing how cross-system dependencies are expressed through governed ORD IDs rather than hard-wired URLs.

System Landscape Model. ORD assumes the system landscape is known before description is harvested. The central concept is the *System Instance*, which corresponds to one tenant and enforces resource isolation. In Listing 1, `describedSystemInstance.baseUrl` pins the document to a specific tenant. Each instance belongs

Listing 2: Base required fields shared by all three resource types (agent, apiResource, dataProduct). `description` is optional per ORD v1.16 but always generated in this benchmark (273/273 resources).

```

1  ordId      string  namespace:type:PascalCaseName:v1
2  title      string  human-readable name
3  shortDescription string  one sentence, max 120 chars
4  description string  2-3 sentences (*optional in spec, always populated here)
5  version    string  "1.0.0"
6  partOfPackage string  ordId of the containing package
7  visibility  enum    public | internal | private
8  releaseStatus enum    active | beta | deprecated | sunset

```

Listing 3: Resource-specific required and semantic fields (Clean-ORD state). Semantic fields are optional in the ORD spec but always populated in the benchmark because they drive five of the six ambiguity-metric dimensions.

```

1  -- agent --
2  required: (base set only)
3  semantic: relatedEntityTypes string[] sap.odm entity type ordIds
4             lineOfBusiness    string[]
5             tags               string[]
6             industry           string[]
7
8  -- apiResource --
9  required: apiProtocol          enum    rest | odata-v4 | odata-v2 | ...
10            resourceDefinitions object  [{type, mediaType, url, accessStrategies}]
11  semantic: exposedEntityTypes  object  [{ordId: "sap.odm:entityType:...:v1"}]
12            lineOfBusiness, tags, industry
13
14  -- dataProduct --
15  required: type                enum    primary | derived
16            category            enum    business-object | analytical | operational | other
17            outputPorts         object  [{ordId}]
18  semantic: entityTypes         string[] sap.odm entity type ordIds
19            lineOfBusiness, tags, industry

```

to a *System Type*, and resources are described relative to a system in the static perspective or a system instance in the dynamic one. This separation matters in two ways. First, identity is anchored in the ORD ID's namespace, so `sap.foo:agent:disputeAgent:v1` remains stable across tenants. Second, the distinction between a live tenant instance and a merely defined system-type capability is preserved, letting an aggregator restrict retrieval to live capabilities.

Namespace ownership and authority namespaces. Every ORD identifier carries a namespace, and the namespace expresses ownership of the definition, not the location where it was found. In Listing 1, `sap.foo` is the *own system namespace* (the publishing system owns its definitions), while `sap.odm`, `sap.aicore`, `sap.joule`, and `sap.bar` each belong to different teams or systems. Three ownership models matter here: an *own system namespace* when a system publishes what it owns, *another system's namespace* when a system re-exposes a contract governed elsewhere, and an *authority namespace* (e.g. `sap.odm`) when a cross-system component governs a contract that multiple systems implement. Authority namespaces let the same governed definition be published from several systems under one identifier instead of being duplicated per publisher. This is critical for retrieval: methods see one resolvable reference regardless of how many publishers exist. The benchmark's shared `sap.odm` entity-type vocabulary (Appendix B.3) is exactly such an authority namespace.

Shared taxonomy, resources and contracts. ORD distinguishes *shared* from *abstract* resources. A shared ORD ID expresses identity: multiple publishers reference the same governed definition. In Listing 1, `sap.odm:entityType:Dispute:v1` appears in both the agent's `relatedEntityTypes` and the API resource's `exposedEntityTypes`, expressing that both operate on the same business object in the ODM namespace. An abstract resource, by contrast, expresses compatibility: one resource defines an interface-only contract, and concrete resources declare they are `compatibleWith` it. The shared-ID pattern is what makes cross-system references resolvable: the integration dependency in the listing points to one governed API contract regardless of which instance implements it. Combined with the System Landscape Model, this keeps cross-resource references (`integrationDependencies`, `exposedApiResources`, `partOfConsumptionBundles`, and entity-type links) resolvable as the landscape grows.

B. Resource Model

This appendix details the ORD field schema every resource in ORD-Bench conforms to, an illustrative example of a resource in its clean and enriched state, and the system and entity-type vocabulary that structures the landscape.

B.1 ORD Field Schema

Before the landscape can be constructed, the schema each resource must conform to has to be defined. The schema decides which fields are available to retrieval methods at design-time and runtime. Each resource in the benchmark conforms to

Listing 4: Enriched-ORD additional fields. Written by `enrich_from_processes.py` for GT-eligible resources only (70/273). Non-GT resources remain at Clean-ORD level.

```

1 capabilities  string[]  verb-noun tokens from process step labels
2                  e.g. ["diagnose-equipment", "schedule-maintenance"]
3 useCases      string[]  user-facing step descriptions (1 sentence each)
4                  e.g. ["Identify root cause of unplanned downtime"]
5 processNext   string[]  ordIds of the next GT-eligible step's resource
6                  creates process-sequence edges for graph traversal
7 partOfGroups  object[]  [{groupId: "proc_014", groupTypeId: "bpmn|"cmmn"}]
8                  links the resource to its parent process model

```

ORD v1.16. Three listings below separate the fields by purpose: the base fields present in all Clean-ORD resources, the resource-specific required and semantic fields, and the enrichment fields that are written only for GT-eligible resources during the design-time flow and are therefore present in Enriched-ORD. Listing 2 shows the base fields shared by all three resource types. All 273 resources carry `description` even though it is optional in the spec.

Listing 3 shows the resource-specific fields. The most important difference between the three types is how they reference business objects. All three carry an entity-type field, but under different names and structures: agents use `relatedEntityTypes` (a flat string array), apiResources use `exposedEntityTypes` (an object array with `ordId` keys), and dataProducts use `entityTypes` (a flat string array). This asymmetry is transparent to the ambiguity metric, which normalises all three to a flat `ordId` set before computing Jaccard similarity. Retrieval methods that traverse the raw ORD graph, however, interact with these fields directly, so any graph-walk entry-point logic must check all three field names when seeding traversal.

Listing 4 shows the four enrichment fields added during the design-time flow. These fields are absent in Clean-ORD and only written for GT-eligible resources. The 203 non-GT resources remain at Clean-ORD level even in the enriched state. This is the experimental variable: the Δ (Enriched – Clean) in the evaluation results measures exactly how much these four fields improve retrieval quality.

B.2 Example ORD Resource: Clean and Enriched State

The listing below shows one benchmark resource (`my.mes:agent:ProductionChangeOrchestrator:v1`) in both states side by side. The left column is the Clean-ORD state as retrieved methods see it; the right column shows the four fields written by the design-time enrichment flow. The `entityTypes` field and all base fields are identical in both states — enrichment only adds, never modifies.

B.3 Systems and Entity Type Vocabulary

The landscape spans 10 enterprise systems from five vendors (SAP, Workday, Emarsys, Siemens, a generic corporate ITSM system), covering the domains most commonly encountered in enterprise orchestration scenarios. All systems share a single entity-type vocabulary of 50 business-object types derived from the SAP One Domain Model (`sap.odm`) — these cross-domain connections had to be established explicitly during landscape construction, since real enterprise systems do not share a unified vocabulary by default. The shared vocabulary creates the cross-system similarity connections that make retrieval non-trivial: a Workday resource referencing `sap.odm:entityType:Employee:v1` is semantically linked to SAP SuccessFactors resources exposing the same type, making cross-namespace disambiguation a genuine retrieval challenge rather than a trivial namespace lookup.

The 50 entity types used are (all prefixed `sap.odm:entityType:...:v1`): Account, Approval, Asset, Benefit, Budget, Campaign, ChangeRequest, Compensation, Compliance, Contract, CostCenter, Customer, CustomerAccount, CustomerJourney, CustomerOrder, CustomerSegment, Document, Employee, EngineeringChange, ExpenseReport, HazardousSubstance, ITEquipment, Incident, Invoice, IoTSensorStream, Location, Machine, MaintenanceOrder, Material, Notification, Payroll, PerformanceReview, ProductItem, ProductionOrder, Project, PurchaseOrder, PurchaseRequisition, QualityInspection, Report, SafetyIncident, SalesOpportunity, ServiceTicket, SoftwareLicense, Substance, Supplier, Task, TimeOff, Vendor, WorkOrder, WorkforcePerson.

C. Ambiguity Metric & Resource Generation

This appendix details how pairwise resource similarity is quantified and how the adversarial generation pipeline uses that metric to build the landscape.

Listing 5: Example resource in Clean-ORD and Enriched-ORD state. Fields absent in the clean state are marked – absent –; the four added fields are written back from the design-time process-matching flow after validation.

```

1  -- CLEAN (ord.json) --                -- ENRICHED (ord_enriched.json) --
2
3  "ordId": "my.mes:agent:ProductionChangeOrchestrator:v1"
4  "title": "Production Change Orchestrator"
5  "version": "1.0.0"
6  "entityTypes": [                      (unchanged)
7    "sap.odm:entityType:EngineeringChange:v1",
8    "sap.odm:entityType:Material:v1",
9    "sap.odm:entityType:ProductItem:v1"
10 ]
11
12 "capabilities": -- absent --           "capabilities": [
13                                           "manage-material-updates",
14                                           "validate-engineering-specs",
15                                           "sequence-work-orders", ...
16 ]
17
18 "partOfGroups": -- absent --           "partOfGroups": [
19                                           { "groupId":
20                                             "proc_ManufacturingChange_v1",
21                                             "groupTypeId": "bpmn"}, ...
22 ]
23
24 "processNext": -- absent --            "processNext": [
25                                           "sap.sf:apiResource:
26                                             CompensationManagement:v1"
27 ]
28
29 "useCases": -- absent --               "useCases": [
30                                           "Operations manager submits
31                                           engineering change order ...",
32                                           ... (8 total)
33 ]

```

C.1 Ambiguity Metric

Without a controlled similarity structure, a randomly generated landscape would consist mostly of structurally unrelated resources, and any retrieval method would identify the correct resource trivially. The ambiguity metric prevents trivial retrieval by quantifying how confusable any two resources are, so the generation pipeline can place genuine distractors next to every GT-eligible resource. The metric composition, formula, and scoring method per dimension are shown in Tab. 2 in the main text; the implementation is in `src/adversarial/preselect.py`.

The six dimensions are weighted equally by design; differential weights would lack an empirical basis since no labelled confusion dataset exists for ORD resources. The cross-namespace bonus and type penalty are theoretically motivated. An embedding-based alternative was excluded because embedding-based retrieval methods evaluated on the benchmark rely on the same representation, and using it for construction would bias the difficulty structure towards them.

The scores translate directly into similarity tiers that govern GT eligibility. The tier boundaries (HIGH ≥ 0.50 , MEDIUM 0.25–0.49, LOW 0.10–0.24) were derived from pilot analysis on the 30 seed resources: a score above 0.50 corresponds to pairs that simultaneously share entity types, line of business, and tags — the combination a human rater also flags as requiring system context to distinguish. Scores of 0.25–0.49 correspond to partial-field matches that require domain context, and 0.10–0.24 to resources sharing at most one or two fields. A resource qualifies as an evaluation target only if it has at least 3 HIGH, 5 MEDIUM, and 5 LOW neighbours, guaranteeing genuine distractors at every confusability level. Raising these thresholds reduces the eligible pool further but does not alter the tier definitions; lowering them admits resources with insufficiently dense neighbourhoods. Table 6 shows one validated pair per tier with the full per-field decomposition.

The extended metric used in the disambiguation analysis (App. E.2) adds three semantic dimensions (`capabilities`, `partOfGroups`, `useCases`); a dimension is counted only when both resources carry it (Tab. 7). The implementation is in `experiments/disambiguation/extended_metric.py`.

C.2 Generation Pipeline and Audit Trail

The previous sections defined what resources must look like (Sec. B.1), which systems and vocabulary they span (Sec. B.3), and how similarity between them is measured (Sec. C.1). This section describes how the landscape was built using those foundations. Resources were generated in three sequential steps: seed generation (`generate_seeds.py`), iterative gap-filling (`generate_iterative.py`), and near-duplicate reduction (`reduce_near_dups.py`). The iterative approach is key: each round adds resources targeted at the remaining similarity gaps of existing resources, so the landscape grows denser until every GT-eligible resource has its required neighbourhood. All LLM calls used `claude-haiku-4.5` at

Table 6: Similarity tier examples. One representative pair per tier from the benchmark landscape. HIGH pairs require system-context disambiguation; MEDIUM pairs require domain-context disambiguation; LOW pairs share only partial structural overlap. Scores decompose into *text* (TF-IDF cosine), *lid* (localId), *ET* (entity types), *LoB* (line of business), and *tags*, plus a +0.5 cross-namespace bonus; *industry* is 0 for all three pairs and omitted. The six dimensions are summed with the bonus and divided by 6 (see Tab. 2). Only *title* is shown per resource; the *text* score is computed over the full *title* + *shortDescription* + *description* and *tags* over the (unshown) tag sets, so those two components are not re-derivable from the displayed rows alone.

Tier	Field	Resource A	Resource B
HIGH	ordId	emarsys.cx:dataProduct: CampaignPerformanceAnalytics:v1	sap.crm:dataProduct: CampaignSegmentationAndROIAnalytics: v1
	title	Campaign Performance Analytics	Campaign Segmentation and ROI Analytics
	system	emarsys.cx	sap.crm
	entityTypes	Campaign, CustomerSegment, CustomerAccount	Campaign, CustomerAccount, CustomerSegment
	lineOfBusiness	Marketing and CX	Marketing and CX
	Score:	text=0.44, lid=0.50, ET=1.00, LoB=1.00, tags=1.00, cross-ns+0.5 → sim=0.74	
MEDIUM	ordId	sap.crm:apiResource: SalesTeamAvailabilityAndCapacityService: v1	workday.hcm:apiResource: TimeOffAndCapacityAllocationManagement: v1
	title	Sales Team Availability and Capacity Service	Employee Absence and Resource Utilization API
	system	sap.crm	workday.hcm
	entityTypes	Employee, TimeOff	Employee, TimeOff
	lineOfBusiness	Sales	Sales
	Score:	text=0.38, lid=0.11, ET=1.00, LoB=1.00, tags=0.00, cross-ns+0.5 → sim=0.50	
LOW	ordId	corp.itsm:apiResource: ProcurementVendorManagementAPI:v1	my.mes:apiResource: MaterialAndVendorMasterData:v1
	title	Procurement Vendor Management API	Material and Vendor Master Data API
	system	corp.itsm	my.mes
	entityTypes	PurchaseOrder, Vendor	Material, Vendor
	lineOfBusiness	Operations	Product Lifecycle Management
	Score:	text=0.31, lid=0.17, ET=0.32, LoB=0.00, tags=0.20, cross-ns+0.5 → sim=0.25	

Listing 6: Generator system prompt, used for both seed and iterative phases. The prompt enforces that all systems reference `sap.odm` entity types and that descriptions reflect resource type through verb choice rather than by naming the type explicitly.

```

1 You are an ORD resource designer for enterprise software benchmarks.
2 Generate ONE realistic ORD resource JSON for the given system and type.
3 ALL systems use sap.odm:entityType:* IDs -- including non-SAP systems.
4 Description must naturally reflect the entity types WITHOUT explicitly naming
5 the resource type:
6   agent      -> action verbs (monitors, automates, coordinates, processes)
7   apiResource -> data verbs (exposes, retrieves, manages, provides access to)
8   dataProduct -> analytical verbs (aggregates, summarizes, provides metrics on)
9 ALL semantic fields must be non-empty. Respond with ONLY valid JSON.

```

temperature 0.0, seed 42, via a local LiteLLM proxy; the file-based cache (keyed on `hash(model, system, prompt)`) makes every accepted output reproducible.

Quality gate (applied identically in Steps 1 and 2). Every generated candidate passes a two-stage gate. The deterministic ORD v1.16 spec validator (`validate_ord.py`) runs first at zero LLM cost as criterion C1: it checks the required fields per resource type and the `ordId` pattern, and a failure returns immediately without an LLM call. Only structurally valid resources reach the LLM Judge (embedded in `generate_seeds.py` and `generate_iterative.py`), which evaluates four semantic criteria: C2 (coherent with the system domain), C3 (not a duplicate of an existing resource), C4 (entity-type references semantically justified), and C5 (description makes domain sense). Listing 6 shows the Generator system prompt and Listing 7 the Judge criteria.

Step 1 — Seed generation (`generate_seeds.py`). 30 seed resources (3 per system) established the starting population. Entity types were assigned explicitly via `ENTITY_TYPES_BY_DOMAIN`, ensuring all 50 `sap.odm` types were referenced by at least one seed. Each candidate was passed through the quality gate described above before being accepted into the landscape.

Step 2 — Iterative gap-filling (`generate_iterative.py`). Starting from the 30 seeds, new resources were generated round by round. Each round filled HIGH gaps before MEDIUM, because HIGH-fill resources sharing the same entity

Listing 7: Judge system prompt from `generate_iterative.py` (iterative phase). The seed phase (`generate_seeds.py`) uses the same C2–C5 criteria with slightly different phrasing; the instruction to accept cross-domain entity types is present in both.

```
1 You are a benchmark quality judge for ORD resources (synthetic ambiguity benchmark).
2 Evaluate C2–C5. ALL systems use sap.odm:entityType:* IDs -- never reject for this.
3
4 CRITICAL: This benchmark intentionally creates cross-domain resources to build
5 ambiguity. Cross-domain entity types are CORRECT AND DESIRED.
6
7 ACCEPT if the resource makes any reasonable sense for the system.
8 ACCEPT cross-domain ET references if ANY connection exists
9   (e.g. HR+Compliance, ITSM+Project).
10 REJECT only if: completely absurd for the system,
11   OR exact functional duplicate (same title+scope).
12 Respond with ONLY a JSON object.
```

Listing 8: Sample `enrichment_log.json` entry. HIGH-tier resource accepted for `emarsys.cx`.

```
1 {
2   "phase": "iterative", "round": 1, "action": "create", "outcome": "accepted",
3   "target_resource": "emarsys.cx:apiResource:JourneyContactStateAPI:v1",
4   "tier_target": "high", "tier_actual": "high", "achieved_sim": 0.5833,
5   "profile": {
6     "et_ids": ["sap.odm:entityType:Campaign:v1", "sap.odm:entityType:CustomerAccount:v1"],
7     "lob": ["Marketing and Customer Experience"],
8     "tags": ["journey-orchestration", "omnichannel", "contact-state", "real-time-engagement"],
9     "rtype": "apiResource", "namespace": "sap.crm"
10  },
11  "solver_breakdown": {
12    "text": 0.0, "localId": 0.0, "entityTypes": 1.0, "lineOfBusiness": 1.0,
13    "tags": 1.0, "industry": 0.0, "same_type": true, "cross_namespace": true,
14    "type_penalty_applied": false, "cross_namespace_bonus_applied": true
15  },
16  "judge_verdict": {"c2_coherent": true, "c3_not_duplicate": true,
17                    "c4_et_justified": true, "c5_description_ok": true,
18                    "accepted": true, "reject_reason": null},
19  "generator_tokens": 1231, "judge_tokens": 1087
20 }
```

types, LoB, and a different namespace often satisfied MEDIUM slots for other resources at the same time, maximising shared coverage. Similarity was computed without TF-IDF during generation (`fast_sim`: `entityTypes`, `lineOfBusiness`, `tags`, `localId`, all $O(1)$ Jaccard), making generation fast. Before each LLM call, a pre-check verified that a resource with the target structural profile (entity types, LoB, resource type) could at all land in the required tier. Concretely: the metric was computed between a dummy candidate carrying only the structural fields known before generation (`entityTypes`, `lineOfBusiness`, `tags`, `localId`; text fields set to empty) and the target resource. Because the only fields the LLM fills in are the free-text ones (`title`, `shortDescription`, `description`, and the resource name in the `ordId`) — all structural fields are pre-determined and fixed by the pre-check profile — a dummy score below the tier threshold means no LLM output could ever produce a valid distractor. The gap was therefore skipped without an LLM call, eliminating wrong-tier failures before they cost tokens.

Step 3 — Near-duplicate reduction (`reduce_near_dups.py`). After generation, a full TF-IDF scan identified all pairs with `full_sim` ≥ 0.75 . For each pair, if one resource already had $H \geq 3$ neighbours, it was removed without loss. Otherwise the resource was rewritten via LLM to lower text similarity below 0.75. This step was necessary because LLM-generated descriptions for structurally similar resources tend to share vocabulary, pushing `full_sim` above 0.75 even when `fast_sim` was below during generation. The full generation log is at `benchmark/landscape/logs/enrichment_log.json`. A representative entry is shown in Listing 8.

D. Test Case Construction

This appendix details the Generator–Judge pipelines and prompts used to produce the design-time and runtime test cases.

D.1 Design-Time Cases

LLM setup. Generation and judging in both C.3.1 and C.3.2 used the same shared LLM client (`src/core/llm.py`): model `claude-haiku-4.5`, temperature 0.0, seed 42, served via a local LiteLLM proxy. Temperature 0.0 makes generation deterministic. Combined with the file-based cache (keyed on `hash(model, system, prompt)`), every accepted output is reproducible. Embeddings used `text-embedding-3-large` via the same proxy.

Process model construction. The script `benchmark/test_cases/design_time/generate_processes.py`

Listing 9: Process Generator prompt (generate_processes.py). Model: claude-haiku-4.5, temp=0.0, seed=42.

```
1 Create a coherent 8-step {BPMN|CMMN} enterprise process model using EXACTLY these resources.
2
3 GT-ELIGIBLE RESOURCES (4 steps, evaluation targets):
4 {gt_block}
5
6 NON-GT RESOURCES (4 steps, supporting roles):
7 {non_gt_block}
8
9 Rules:
10 - Exactly 8 steps, one step per resource, in logical enterprise order
11 - Activity labels: business activities (e.g. "Diagnose equipment failure"), NOT resource names
12 - Activity descriptions: describe the business need, NOT the resource type
13 - The process must tell a coherent real enterprise scenario
14 - For CMMN: use case-driven/conditional structure; for BPMN: sequential/structured
15
16 Output TWO parts:
17 PART 1 - Process XML:
18 <process id="proc_[short_name]_v1" type="{process_type}">
19   <step id="s1" label="[activity label]"
20     description="[business need in 1 sentence]"
21     ordId="[ordId]"
22     capability="[verb-noun, e.g. diagnose-equipment]"
23     useCase="[1 sentence user-facing context]" />
24   ...
25   <sequenceFlow from="s1" to="s2" />
26 </process>
27
28 PART 2 - Enrichment (GT resources only, JSON):
29 {
30   "gt_ordId": {
31     "capabilities": ["verb-noun"],
32     "useCases": ["1 sentence"],
33     "processNext": ["[ordId_of_next_gt_step]"],
34     "partOfGroups": [{"groupId": "proc_[name]_v1", "groupId": "{process_type}"}]
35   }
36 }
```

Listing 10: Process Judge system + user prompt. J1–J5 must all pass; J6 (enrichment quality) is evaluated but not blocking.

```
1 SYSTEM: You are a quality judge for a synthetic enterprise benchmark. Be pragmatic:
2 accept processes that are plausible and well-structured, even if imperfect.
3 Reject only clear failures. Answer ONLY with valid JSON.
4
5 USER:
6 Review this {BPMN|CMMN} process model for a retrieval benchmark.
7
8 PROCESS XML: {xml_text[:4000]}
9 ENRICHMENT: {json.dumps(enrichment)[:1500]}
10
11 Rate each criterion -- be generous, this is a synthetic benchmark:
12 {
13   "J1": true/false, // process type fits: BPMN=ordered steps, CMMN=adaptive
14   "J2": true/false, // step labels describe business activities (not "call API X")
15   "J3": true/false, // step descriptions state a business need (not "this agent does X")
16   "J4": true/false, // resources are plausibly matched to their activities
17   "J5": true/false, // steps together form a coherent enterprise scenario
18   "J6": true/false, // enrichment fields are semantically correct
19   "accepted": true/false, // true if J1+J2+J3+J4+J5 all pass (J6 optional)
20   "reason": "one sentence only if rejected"
21 }
```

builds the 30 process models. Resource selection is deterministic: each model uses exactly 4 GT-eligible and 4 non-GT resources, spanning at least 3 namespaces with at least one agent and one apiResource. GT resources are chosen with coverage awareness so that all 70 GT-eligible resources appear across the 30 models ($30 \times 4 = 120 \text{ slots} \geq 70$). The 4/4 split is deliberate: GT steps face genuine HIGH-tier distractors that make retrieval non-trivial, while non-GT steps fill supporting roles. The Generator produces process XML and enrichment fields in a single call (Listing 9). A deterministic Validator (V1–V6, zero LLM cost) checks structural rules first; the LLM Judge (J1–J6) then verifies semantic coherence only for outputs that passed. The Judge accepts plausible processes even if imperfect and rejects only clear failures. J6 checks enrichment field quality but is not blocking (Listing 10). The 30 accepted models are a fixed input to the design-time evaluation flow.

SKILL.md construction. The script `benchmark/test_cases/design_time/generate_skills.py` generates one SKILL.md per accepted process model from the same process XML. The Generator writes the SKILL.md in one pass (Listing 12), and a deterministic Validator (no LLM call) then enforces four structural rules: S1 requires exactly one `ord_confirmed` `ordId` per step. S2 requires that every such `ordId` exists in the 273-resource landscape. S3 forbids

Listing 11: Representative SKILL.md artefact (proc_023, CMMN, first 4 of 8 steps). GT-eligible steps carry an ord_confirmed comment pinning the verified ordId; non-GT steps do not.

```

1  ---
2  name: proc_023
3  description: >
4      This skill orchestrates production transitions triggered by engineering
5      changes, coordinating workforce readiness, equipment validation, asset
6      provisioning, and customer communications.
7  metadata:
8      process-id: proc_023  process-type: cmmn
9      landscape-version: v2-benchmark-273
10 ---
11 ## Steps
12 ### Step 1: Analyze Engineering Change Impact and Workforce Readiness
13 <!-- ord_confirmed: sap.s4:dataProduct:EngineeringChangeImpactAndWorkforceReadiness:v1 -->
14 **Input:** Engineering change request, current workforce roster
15 **Output:** Impact assessment report, workforce readiness scorecard
16 **Capability:** analyze-change-impact
17 Evaluate how the engineering change affects production operations and
18 identify skill gaps. Plant managers use this to determine staffing
19 adjustments before proceeding with the transition.
20
21 ### Step 2: Validate Material and Equipment Specifications
22 **Input:** Engineering change specifications, current material definitions
23 **Output:** Validated material list, compatibility report
24 **Capability:** validate-specifications
25 Retrieve and verify all material definitions and vendor specifications
26 impacted by the engineering change.
27
28 ### Step 3: Assess Equipment Service Readiness and Dependencies
29 <!-- ord_confirmed: my.mes:apiResource:EquipmentServiceAndChangeManagement:v1 -->
30 **Input:** Equipment inventory, service history, dependency map
31 **Output:** Equipment readiness assessment, service recommendations
32 **Capability:** assess-equipment-readiness
33 Evaluate equipment condition and service status, identifying cascading
34 dependencies across the manufacturing line.
35
36 ### Step 4: Orchestrate Asset Provisioning for Transition
37 <!-- ord_confirmed: corp.itsm:agent:AssetProvisioningOrchestrator:v1 -->
38 **Input:** Asset requirement list, production transition schedule
39 **Output:** Asset allocation plan, provisioning order
40 **Capability:** orchestrate-asset-provisioning
41 Coordinate allocation and provisioning of materials and equipment needed
42 for the production transition.

```

duplicate ordIds within the same skill. S4 requires that step descriptions mention the business activity but not the resource name or ordId. A skill is accepted once S1–S4 pass. These deterministic structural checks proved sufficient for the SKILL.md format, so no LLM judging step is applied here. The ord_confirmed annotation is an HTML comment that embeds the ordId confirmed at design time. Listing 11 shows the first four steps of proc_023 (CMMN), illustrating how the confirmed ordIds are embedded while step descriptions remain system-agnostic. The generated skills are stored once and not regenerated on each design-time run, keeping the ground truth consistent across methods.

ORD enrichment. The script benchmark/test_cases/design_time/enrich_from_processes.py extracts enrichment fields deterministically from the process XML without an LLM call. For each GT-eligible step it reads the capability and useCase attributes written by the Generator, derives processNext from the sequence flow, and writes partOfGroups from the process identity. These four fields are then written back into the ORD document of the corresponding GT-eligible resource, producing the Enriched-ORD state. Non-GT steps are skipped. Appendix B.2 shows a resource in its clean and enriched states with the four added fields. Like the skills, the enriched ORD documents are generated once and reused unchanged across all evaluation runs.

Logs. Every construction attempt is logged. Skill attempts (validator results, outcome, generator tokens) go to benchmark/test_cases/design_time/logs/skill_construction_log.json; process-model and enrichment attempts are logged alongside it in process_construction_log.json and enrichment_log.json. Listing 13 shows one entry from each.

D.2 Runtime Cases

The 110 runtime cases span four orchestration modes: Skill-Guided ($n = 30$), Skill-Adjusted ($n = 20$), Dynamic ($n = 40$, 20 single-intent and 20 multi-intent), and Out-of-Scope ($n = 20$). All are built by the same three-party loop used for design-time cases (Generator → deterministic Validator → LLM Judge, claude-haiku-4.5, temp=0.0, seed=42). A single-call LLM Baseline-Solver acts as a difficulty gate, where a test case is regenerated (up to ten times) until the solver fails

Listing 12: SKILL.md Generator prompt.

```
1 Create a SKILL.md file for process '{process_id}' ({BPMN|CMMN}).
2
3 Process steps: {steps_block}
4 GT-eligible ordIds (use ONLY these for ord_confirmed): {gt_ids}
5
6 Rules:
7 - Include ALL steps; add <!-- ord_confirmed: <ordId> --> ONLY for GT-eligible steps
8 - Step descriptions: user-facing business language
9 - Do NOT mention resource names, ordIds, or system names in descriptions
10 - Include realistic Input/Output and Capability per step
11
12 Output the complete SKILL.md:
13 ---
14 name: {process_id}
15 description: > [2-3 sentence scenario description]
16 metadata:
17   process-id: {process_id} process-type: {ptype}
18   landscape-version: v2-benchmark-273
19 ---
20 ## Steps
21 ### Step 1: [label]
22 <!-- ord_confirmed: [ordId] -->
23 **Input:** ... **Output:** ... **Capability:** [verb-noun]
24 [1-2 sentence user-facing description]
```

Listing 13: One representative entry from each of the three design-time construction logs. The skill and process logs record per-attempt validator results and acceptance outcome; the enrichment log records which fields were written per resource.

```
1 -- process_construction_log.json --
2 { "process_id": "proc_001", "attempt": 1, "process_type": "bpmn",
3   "resource_pool": {"gt": [4 ordIds], "non_gt": [4 ordIds]},
4   "generator_tokens": 2604,
5   "validator_results": {"V1": true, ..., "V6": true},
6   "judge_results": {"J1": true, ..., "J6": true},
7   "outcome": "ACCEPTED" }
8
9 -- skill_construction_log.json --
10 { "skill_id": "proc_010", "attempt": 1, "generator_tokens": 1955,
11   "validator_results": {"S1": true, "S2": true, "S3": true, "S4": true},
12   "validator_failure": null,
13   "outcome": "ACCEPTED" }
14
15 -- enrichment_log.json --
16 { "ordId": "corp.itsm:agent:AssetProvisioningOrchestrator:v1",
17   "namespace": "corp.itsm",
18   "fields_added": ["capabilities", "useCases", "processNext", "partOfGroups"] }
```

on it, so only hard cases are kept. The accepted cases are stored under `benchmark/test_cases/runtime/output/` as one JSON file per mode. The per-mode generation is detailed below.

Skill-Guided. Generated by `benchmark/test_cases/runtime/generation/generate_skill_guided.py`. The Generator receives the GT-eligible resources of the matched skill and produces a user request describing the business scenario without naming any resource, system, or ordId. Five judge criteria (C1–C5) check that the prompt is realistic, coherent with the process type, and free of technical identifiers; all five must pass (Listing 14).

The SG prompts were later shortened: the initial verbose requests (avg. 470 characters) enumerated sub-activities that the planner misread as gap steps and routed to `skill_adjusted`, so all 30 were rewritten to focused variants (avg. 168 characters), preserving `skill_id` and `expected_ordIds`. The original prompts remain in the provenance logs as `accepted_case.user_prompt`, the evaluated ones as `post_gate_rewrite.final_user_prompt`. Since the SG metric is routing accuracy rather than retrieval difficulty, the rewrite does not affect the evaluation.

Skill-Adjusted. Generated by `benchmark/test_cases/runtime/generation/generate_skill_adjusted.py`. The Generator produces one coherent request where the matched skill covers the core flow but the gap resources are only implied, never named. Three judge criteria check that the skill covers the core (C1), the gaps are implied not stated (C2), and the scenario is coherent (C3). The solver gate (run over the gap resources) rejects an iteration as `TOO_EASY` when the solver already recovers a gap resource, and on each retry the Generator hint is strengthened to phrase the gap dependency more implicitly (Listing 15).

Dynamic. Generated by `benchmark/test_cases/runtime/generation/generate_dynamic.py`. Each single-intent case exercises one of three problem types recorded in `problems_exercised`: P1 (Agent Sprawl) pairs the ground-truth resource with a HIGH-tier distractor in a different namespace, P2 (Vocabulary Drift) with one sharing the same

Listing 14: SG Generator + Judge prompts.

```
1  -- GENERATOR --
2  Create a natural-language enterprise user request for a {BPMN|CMN} skill.
3
4  The skill covers these activities (DO NOT mention directly):
5  {gt_block}
6
7  Rules: Sound like a real enterprise user ("We need to...", "Help us...").
8  Do NOT name any resource, system, ordId, or skill name.
9  Make it specific enough to suggest a multi-step process. 2-4 sentences.
10 Output ONLY the user prompt text.
11
12 -- JUDGE SYSTEM --
13 You are a quality judge for a synthetic enterprise benchmark.
14 Be pragmatic -- accept prompts that are plausible and well-structured.
15 Answer ONLY with valid JSON.
16
17 -- JUDGE USER --
18 Review this {BPMN|CMN} skill-guided case.
19 user_prompt: {prompt}
20 expected GT ordIds: {expected_ord_ids}
21 {
22   "C1": true/false, // prompt sounds like real enterprise request
23   "C2": true/false, // prompt coherent with a multi-step process scenario
24   "C3": true/false, // no resource names or skill names in prompt
25   "C4": true/false, // skill could plausibly cover request end-to-end
26   "C5": true/false, // prompt has natural language complexity (not trivially obvious)
27   "accepted": true/false, "reason": "one sentence if rejected"
28 }
```

Listing 15: SA Generator + Judge prompts. The mutation hint is strengthened on each retry (synonym vocabulary, then implicit references).

```
1  -- GENERATOR --
2  Create a user request covering a {BPMN|CMN} skill scenario AND additional
3  implicit steps.
4
5  Skill scenario: {skill_description}
6  Additional steps (DO NOT name directly):
7  {gap_block}
8
9  Rules: Main request should match the skill scenario. Additional steps implied
10 by context ("and we also need to...", "including...", "as well as...").
11 Do NOT name any resource, API, system, or ordId. 3-5 sentences total.
12 {mutation_hint}
13 Output ONLY the user prompt text.
14
15 -- JUDGE --
16 {
17   "C1": true/false, // base skill covers the core request (not the gaps)
18   "C2": true/false, // gap dependencies are implied, not stated explicitly
19   "C3": true/false, // overall request sounds coherent as one enterprise scenario
20   "accepted": true/false, "reason": "one sentence if rejected"
21 }
```

entity types, and P3 (Context Sensitivity) has no distractor and describes the need in terms that avoid the resource's own vocabulary. Multi-intent cases pass two to three functionally independent resources that one request must imply. The Solver gate runs on every case, regenerating with progressively more implicit variants until S fails to rank the ground truth first on `gt_ids[:1]` (logged as `too_easy` regenerations). Some ground-truth resources have no close match, so S always ranks them first no matter how the prompt is worded. For these, after ten tries the case is accepted anyway and the built-in HIGH-tier distractor is recorded as the resource a solver would most plausibly confuse them with. Three judge criteria verify realism (C1), absence of resource or system names (C2), and that the request matches its problem-type pattern (C3) (Listing 16).

Out-of-Scope. Generated by `benchmark/test_cases/runtime/generation/generate_out_of_scope.py`. This mode has no ground-truth resource, so instead of a Solver gate an absence check runs on the generated prompt: it rejects the prompt if its token-overlap (Jaccard) similarity exceeds 0.85 against any resource, which also spares the Judge call when the prompt is too close to the landscape. The Judge then checks that the request is realistic (C1) and that the capability is genuinely absent from the landscape systems (C2). For example, `oos-01` requests supplier credit-rating monitoring, which no resource covers.

Logs. Every case evolution is logged to `benchmark/test_cases/runtime/logs/provenance/<case_id>.json`, recording each iteration's Generator output, Solver verdict, and Judge response. Listing 17 shows an example.

Listing 16: DY Generator + Judge prompts.

```
1  -- GENERATOR --
2  Create a {single|multi}-intent enterprise user request for these resources ({P-label}):
3  {gt_block}
4  {distractor_hint}
5  Hint: {p_label_hint}
6  Rules: Do NOT name any resource, API, system, or ordId.
7  Sound like a real enterprise user. 2-3 sentences.
8  {mutation}
9  Output ONLY the user prompt text.
10
11 -- JUDGE --
12 {
13   "C1": true/false, // prompt sounds like real enterprise request
14   "C2": true/false, // no resource name or system name in prompt
15   "C3": true/false, // request matches the {P-label} pattern plausibly
16   "accepted": true/false, "reason": "one sentence if rejected"
17 }
```

Listing 17: Provenance log for dy-32 (multi-intent). The solver first recovers a ground-truth resource (`too_easy`), so the prompt is regenerated until S is diverted to a sibling resource.

```
1  {
2    "case_id": "dy-32", "mode": "dynamic",
3    "query_class": "multi_intent",
4    "selected_resources": [
5      "my.mes:apiResource:ServiceTicketChangeRequestManagement:v1",
6      "sap.ariba:apiResource:MaterialAndVendorMasterDataAPI:v1"
7    ],
8    "evolution_log": [
9      {
10        "iteration": 1, "outcome": "TOO_EASY",
11        "solver_output": {"predicted":
12          "my.mes:apiResource:ServiceTicketChangeRequestManagement:v1"}
13      },
14      {
15        "iteration": 2, "outcome": "ACCEPTED",
16        "generator_output": {"user_prompt": "I need to pull together our
17          current inventory specifications alongside the vendor details we
18          have on file, and simultaneously check on the status of our
19          pending maintenance requests and any open change orders..."},
20        "solver_output": {"predicted":
21          "my.mes:apiResource:MaterialAndVendorInventoryManagement:v1"},
22        "judge_results": {"C1": true, "C2": true, "C3": true}
23      }
24    ]
25  }
```

E. Results & Disambiguation Analysis

This appendix presents the landscape composition and post-hoc embedding validation (E.1), and the extended ambiguity analysis comparing clean and enriched resource descriptions (E.2).

E.1 Generation Results

The construction pipeline yields the 273-resource landscape shown in Fig. 7: ten enterprise systems whose resources cluster around the shared ODM entity types that bridge namespaces, producing exactly the confusable cross-system neighbourhoods retrieval must disambiguate. The resource type distribution (50% `apiResource`, 25% `agent`, 25% `dataProduct`) reflects real enterprise API landscapes, which are dominated by REST APIs while agents and analytical data products are less common. Tab. 4 shows the final composition. GT-eligibility (26% of resources) is not uniformly distributed: `sap.ehs` and `sap.sf` have fewer GT-eligible resources because their entity types (safety incidents, HR objects) are less cross-domain than, e.g., `corp.itsm` or `my.mes`, which share business objects (Vendor, Material, WorkOrder) with multiple other systems.

A design decision behind the ambiguity metric (Appendix B) was to score similarity from ORD fields alone rather than from embeddings, both to keep every tier assignment auditable and to avoid handing embedding-based retrieval a difficulty structure defined in its own representation. To validate this, each of the 273 resources was embedded with `text-embedding-3-large` using its `title`, `shortDescription`, and `description`, and the embedding cosine of all $\binom{273}{2} = 37,128$ pairs was compared against the structural ambiguity score. Embeddings were computed after landscape construction and do not influence it. The analysis runs via `experiments/embedding_analysis/scripts/embedding_vs_metric.py` and `experiments/embedding_analysis/scripts/embedding_by_tier.py`; outputs are in `experiments/embedding_analysis/`.

Fig. 8(a) groups all 37,128 resource pairs by structural tier and shows a clear monotonic pattern: HIGH-tier pairs reach a

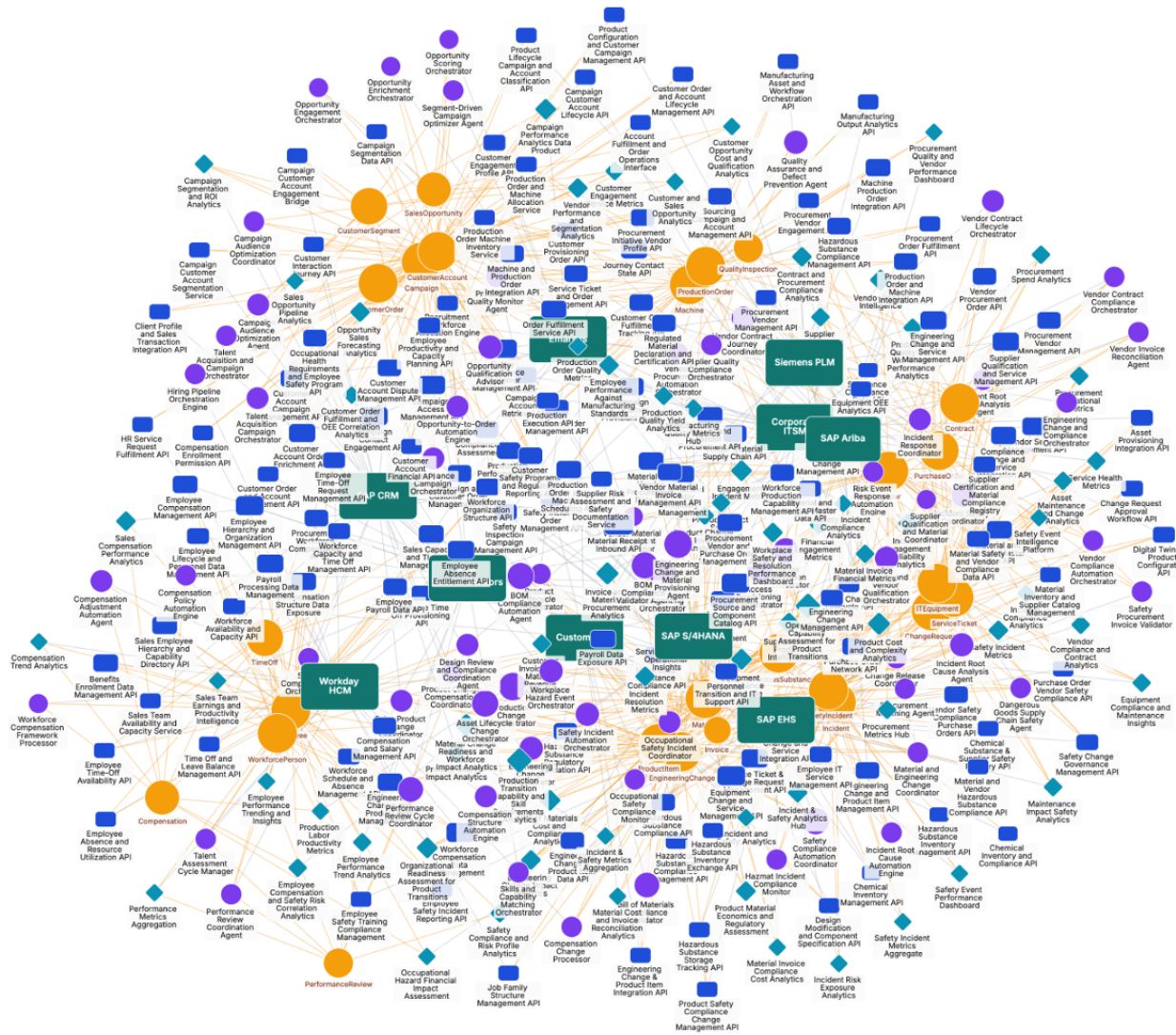


Figure 7: The 273-resource landscape across 10 systems (green nodes). Resources are typed — API resources (blue squares), agents (purple circles), data products (diamonds) — and orange nodes are shared ODM entity types. Edges connect resources through shared entity types, process groups, and process-sequence links. Dense connectivity around shared entity types (e.g. Material, Compensation) creates the cross-namespace distractors the ambiguity metric quantifies and the generation pipeline deliberately amplifies.

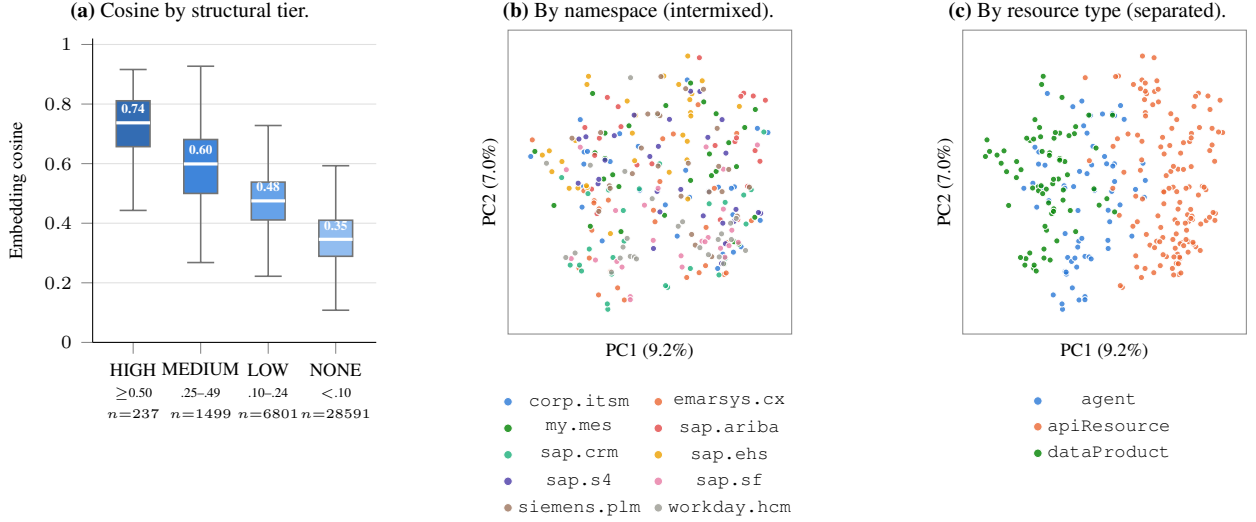


Figure 8: Post-hoc embedding validation on the 273-resource landscape. **(a)** Embedding cosine rises monotonically with the structural ambiguity tier (median 0.74 / 0.60 / 0.48 / 0.35 for HIGH/MEDIUM/LOW/NONE): pairs the field-based metric marks as confusable are independently confusable to an embedding model. **(b)** In PCA-projected embedding space the ten namespaces are thoroughly intermixed, with no system forming its own cluster. Resources from different systems sit close together when they serve the same business function. **(c)** Resource types form distinct regions, consistent with the metric’s cross-type penalty. PC1+PC2 capture only 16% of variance (embeddings are 3072-dimensional), so the projection understates the separation.

median embedding cosine of 0.74, MEDIUM 0.60, LOW 0.48, and unrelated (NONE) 0.35. What the structural metric marks as confusable is also confusable to an embedding model. Across all pairs, the two similarity scores correlate at $r = 0.63$ (Pearson, linear agreement) and $\rho = 0.62$ (Spearman, rank agreement), where 1.0 would mean perfect agreement and 0 none. A value near 0.63 means the two measures agree on the overall trend but are not interchangeable. This gap is intentional: the structural metric penalises cross-type pairs and rewards cross-namespace ones, which embedding cosine does not. The benchmark difficulty is therefore not simply a restatement of an embedding retriever’s own signal, which keeps the comparison across methods fair.

Projecting the 273 embeddings onto two dimensions with PCA shows two further properties. When coloured by system (Fig. 8(b)), the ten namespaces are fully mixed: resources group by what they do, not by which system they belong to. When coloured by resource type (Fig. 8(c)), a clear split appears, with `apiResource` in its own region apart from `agent` and `dataProduct`. This matches the metric’s cross-type penalty. The two shown dimensions capture only about 16% of the total variation, so the real separation is even stronger than the plot suggests.

E.2 Disambiguation Analysis

This appendix details how the semantic enrichment fields affect pairwise resource ambiguity and how that effect compares to embedding similarity. On the clean landscape, structural ambiguity metric and cosine similarity broadly agree ($r = 0.63$, App. E.1, Fig. 8). To evaluate whether enrichment reduces ambiguity between confusable resource pairs, the base metric is extended with the three semantic fields. The analysis in Sec. 4 argues that ambiguity is a property of the representation, not of the resources; the results here confirm this: structural ambiguity falls while embedding cosine rises on the same pairs.

Extended ambiguity metric. The benchmark’s ambiguity metric (Sec. 3, App. B) scores each resource pair over six ORD fields (`text`, `localId`, `entityTypes`, `lineOfBusiness`, `tags`, `industry`). For this analysis it is extended with three design-time semantic fields: `capabilities` and `partOfGroups` scored by Jaccard overlap, and `useCases` scored by TF-IDF cosine. The field `processNext` is excluded because it encodes directed process ordering between resources rather than a property of a single resource. A semantic field enters the score only when both resources in a pair carry it, so the denominator grows from 6 to at most 9. This ensures a score can only fall when two resources actively describe different capabilities or use cases, not as an artefact of a larger denominator. Tab. 7 lists all nine dimensions.

Tab. 7 applies the extended metric to one HIGH-tier pair. `AssetProvisioningOrchestrator` and `AssetLifecycleChangeOrchestrator` share entity types, line of business, tags, and text, giving a clean structural score of 0.75. Their capabilities and process groups are disjoint, so adding semantic fields lowers the score to 0.53. The embedding moves in the opposite direction: cosine rises from 0.79 to 0.82 because the added text increases token overlap.

Populations and mean ambiguity. The analysis runs on the 70 resources that carry all semantic fields, giving 2415 unordered pairs where both sides are enriched and the extended metric can act on both. Fig. 4 reports the mean pairwise

Table 7: Extended ambiguity metric: nine dimensions with scoring method, an excerpt of the actual field contents of each resource, and the resulting sub-score for the AssetProvisioningOrchestrator (A, corp.itsm) vs AssetLifecycleChangeOrchestrator (B, sap.s4) pair. Identical tags give 1.00, but the disjoint capabilities and partOfGroups give 0.00. The three semantic fields are added only when both resources carry them; the denominator grows from 6 to 9 only for those active dimensions. Structural ambiguity therefore falls from 0.75 to 0.53, while embedding cosine rises from 0.79 to 0.82.

Dimension	Scoring	Resource A (excerpt)	Resource B (excerpt)	Score
<i>Original six fields</i>				
text	TF-IDF cosine, title+desc	“Asset Provisioning Orchestrator”	“Asset Lifecycle Change Orchestrator”	0.60
localId	Jaccard, CamelCase tokens	AssetProvisioningOrchestrator	AssetLifecycleChangeOrchestrator	0.40
entityTypes	IDF-weighted Jaccard	(none)	(none)	1.00
lineOfBusiness	Jaccard	it-service-management	it-service-management	1.00
tags	Jaccard	asset-lifecycle, change-automation, provisioning-orchestration	asset-lifecycle, change-automation, provisioning-orchestration	1.00
industry	Jaccard	(none)	(none)	0.00
<i>Semantic extension</i>				
capabilities	Jaccard, verb-noun tokens	orchestrate-asset-provisioning, allocate-materials, ...	orchestrate-engineering-change, manage-change-approvals, ...	0.00
partOfGroups	Jaccard, group IDs	ProductTransitionSafetyCoordination, ...	product_launch_qa, ChangeTalentIntegration	0.00
useCases	TF-IDF cosine	“provisions change-related equipment to production floor”	“coordinate approvals from manufacturing, supply chain, HR”	0.29
Cross-ns bonus	+0.5 cross-namespace	corp.itsm vs sap.s4 (cross-namespace)		+0.5
Type penalty	×0.5 if types differ	both agent (same type)		×1.0
Formula: $\text{sim}^+(a, b) = (\sum_{d \in D \cup D_{ab}^+} s_d + 0.5_{\text{cross-ns}}) / (6 + D_{ab}^+)$ Structural ambiguity: Clean ($ D =6$) 0.75 → Enriched ($ D =9$) 0.53 Embedding cosine (text-embedding-3-large): Clean 0.79 → Enriched 0.82				

ambiguity under the original versus the extended metric for two groups: all 2 415 enriched pairs ($0.111 \rightarrow 0.084$, -24.5%) and the 85 of them that were HIGH-tier ($\text{sim} \geq 0.50$) under the original metric ($0.643 \rightarrow 0.461$, -28.3%). The HIGH-tier subset isolates the pairs that actually risk confusion and shows the larger relative drop: semantic enrichment reduces structural ambiguity most where it matters.

Embedding contrast on the same pairs. Embedding cosine moves opposite to the structural drop. Embeddings are computed with text-embedding-3-large. The clean text concatenates title, shortDescription, and description, while the enriched text appends partOfGroups, processNext, capabilities, and useCases separated by the | character. Fig. 6 reports cosine for three pair types. For all 2 415 both-enriched pairs the mean cosine *rises* from 0.392 to 0.501 ($+27.7\%$). For the 85 previously HIGH-tier pairs it rises from 0.720 to 0.761 ($+5.7\%$). For the 14 210 mixed pairs where only one resource is enriched it rises from 0.368 to 0.388 ($+5.3\%$). The mixed case is the clearest one. The structural ambiguity metric leaves such a pair unchanged because the semantic dimensions need both sides, yet enriching one resource alone shifts its vector toward the shared topical region and raises cosine.

The two directions are not a contradiction. A metric that lowers ambiguity and an embedding that raises similarity on the same pairs look contradictory at first, but they are not. Ambiguity is not an absolute property of two resources; it is a property of how a retrieval method represents them, and the two representations genuinely disagree about whether these pairs are easy to tell apart. They move in opposite directions for the same underlying reason. The structural metric compares each semantic field on its own, so two resources with disjoint capability sets score zero on that dimension and are pushed apart. The embedding instead compresses every field into one vector, so appending more topically related text moves both vectors toward the same region and raises cosine similarity in all three pair types above, including mixed pairs where only one resource gains fields. Enrichment therefore lowers ambiguity only for methods that read the typed fields as discrete signals (namespace, entity-type membership, graph edges, capability filters); a method that retrieves purely by embedding distance sees the opposite effect. This structural alignment between retrieval mechanism and enriched field type is the central qualification on the assumption that richer descriptions uniformly improve selection. The full computation is implemented in experiments/disambiguation/.